AUSTRALIAN NATIONAL UNIVERSITY

COMP8620 Part B 完整深度课程讲义

从 MDP 数学建模到风险感知 MCTS、Python 实现与研究型实验

保留第一、二讲完整讲解 · 新增第三至第八讲

课件理论·完整公式·关键代码·研究实验·口试问答

基于 COMP8620 课程材料及 GitHub lrlorry/8620 的逐讲讲义

2026 年 10 月 24 日 · 代码参考 commit 3a6f4de05

目录

第一讲：从机器人随机运动的数学本质出发，理解 B1 —MDP Modelling（3 分） 3
先谈 Claude 所说的“理论上限” . . . 3
整个 Part B 的知识地图 . . . 3
1. 首先，机器人到底在解决什么? . . . 4
2. 为什么用 MDP? . . . 4
3. State：为什么状态不仅包含坐标? . . . 5
4. Action：为什么是联合动作? . . . 6
5. Transition：这是 B1 的数学核心 . . . 7
6. Reward：规划器究竟在优化什么? . . . 9
7. 最终目标：不是走最短路，而是最大化期望累计奖励 . . . 10
第一讲总结：你要建立的思维模型 . . . 10

第二讲：从 Bellman Equation 到 MCTS 12
一、Bellman 方程：为什么不能只考虑下一步? . . . 12
二、Bellman 方程已经有了，为什么还需要 MCTS? . . . 13
三、MCTS 的本质：通过模拟，学习哪些动作值得继续探索 . . . 15
四、第一阶段：Selection —怎么决定继续探索哪个动作? . . . 17
五、第二阶段：Expansion —为什么不能展开所有联合动作? . . . 18
六、第三阶段：Simulation —为什么还需要 Rollout? . . . 20
七、随机结果：为什么同一个动作还会有多个子节点? . . . 22
八、第四阶段：Backpropagation —模拟结果怎样更新 Q? . . . 23
九、整套 MCTS 在 `_search()` 中怎样串起来? . . . 24
十、最容易忽视的地方：搜索结束后，为什么不直接执行 Q 最大的动作? . . . 25
十一、最后：Online Planner 怎么控制时间? . . . 26
第二讲总结 . . . 26

第三讲：B2 的工程灵魂——BFS、Risk-aware Dijkstra 与 `_proposal()` 29
一、先建立三个不同的“距离”概念 . . . 29
二、为什么能够从已知运动模型估计风险? . . . 30
三、`_options()`：每架无人机如何给候选动作打分? . . . 32
四、`_proposal()`：一架一架生成动作，怎么实现集中式协调? . . . 32
五、Greedy、Sample、Explore 为什么要共存? . . . 33
六、`_leaf()`：搜索看到的不是完整未来，怎么办? . . . 33
七、一个深入的反例：为什么 Greedy 很强，MCTS 却可能犹豫? . . . 34

第四讲：B3——从 `plan()` 到 `step()` 的 Python 闭环系统（8 分） 35
一、首先区分三种程序角色 . . . 35
二、一次 Episode 在代码中怎么走? . . . 35
三、为什么代码中的 `simulate()` 还要知道剩余时间? . . . 36
四、10 秒规则不是一句口号，而是一个接口契约 . . . 36
五、为何每个真实时间步都重建搜索根节点? . . . 37
六、从日志读取真实算法行为 . . . 37
七、`--visualize` 与科研图表分别能看到什么? . . . 38
八、B3 的正确性还依赖哪些边界条件? . . . 38
第四讲总结 . . . 38

第五讲：B4（20 分）——怎样像研究人员一样设计实验？ 39
一、从一个科学问题开始，而不是从一张图开始 . . . 39
二、S1：为什么采用 3^3 全因子实验？ . . . 39
三、S2：为什么要做额外控制实验？ . . . 40
四、S3：为什么消融实验比“参数表”有用？ . . . 41
五、S4：增加搜索时间，性能一定提高吗？ . . . 41
六、S5：为什么还要生成新地图？ . . . 42
七、3,300 次 Episode 具体从哪里来？ . . . 42
八、科研统计的核心：为什么要有 95% Confidence Interval？ . . . 42
九、为何不能只比较完成步数？ . . . 43
十、实验有两套不同的“预算协议” . . . 44
第五讲总结 . . . 44

第六讲：B4 的真实结果——怎样正确解释“有效”与“无效”？ 45
一、S1：三个复杂度因素的真实主效应 . . . 45
二、S3：真正的算法贡献在哪？ . . . 46
三、S4：更多搜索是否带来更多收益？ . . . 48
四、S5：四机强噪声为什么尤其困难？ . . . 48
五、补充压力测试：墙壁缺口与共享走廊 . . . 48
六、你应该怎样写出有说服力的结论？ . . . 49
第六讲总结 . . . 49

第七讲：理论上限、失败机制与可以证明的结论 50
一、成功率的最优策略，与奖励的最优策略并不一样 . . . 50
二、如何在数学上给成功率最优值下定义？ . . . 50
三、为什么狭窄通道可能具有不可避免风险？ . . . 50
四、单步概率本身还有“方向不敏感”的微妙结构 . . . 51
五、把三类困难场景分开理解 . . . 51
六、理论上不可能，与算法没找到好策略，不是一回事 . . . 52
七、如何用课程内方法继续研究？ . . . 52
第七讲总结 . . . 52

第八讲：把整份 Part B 讲给考官听——代码地图、口试问答与知识结构 53
一、把整个项目压缩成一张知识结构图 . . . 53
二、最重要的代码阅读地图 . . . 53
三、15 个最值得准备的口头问答 . . . 54
四、现场 Demo 的 90 秒解释框架 . . . 55
五、12 条公式索引：一眼看到知识主线 . . . 56
六、六道理解检验题（附答案提示） . . . 57
第八讲总结 . . . 57

附录 A：原题要求与课件定位 58

附录 B：完整版学习检查清单 58

关于“原封不动”的整理说明：第一讲和第二讲按照此前对话中的正文、公式、代码片段、例子与讲解顺序收录，不做压缩式摘要；原本的交互式滑块与网页组件转成可打印的静态图表，Markdown 表格和流程图保留信息。第三讲起为在相同讲授风格下补全的**新讲次**。正式作业数学定义和实验数值均以附录列出的 `a2.pdf`、课程课件及指定 GitHub 版本为依据；讲义中的新例子、比喻和推论会明确说明。

阅读路线：第一讲 B1（是什么）→ 第二讲 B2 课堂理论（为什么）→ 第三讲启发式（怎么做）→ 第四讲代码执行（怎么跑）→ 第五讲研究设计（怎么检验）→ 第六讲实验分析（得出了什么）→ 第七讲局限性与理论上限（能证明什么）→ 第八讲口试与公式索引（能否解释给别人）。

第一讲：从机器人随机运动的数学本质出发，理解 B1 —MDP Modelling（3 分）

对应 Week 5《MDP Solving》、作业 `a2.pdf` Part B Q1、报告 `A2/partB.tex` 与仿真器 `A2_code/multi_drone.py`。

我们从最基础的思想开始，不急着讲 MCTS 的四个阶段。

我希望你最终不仅知道代码在做什么，还能回答三个更深的问题：

1. **为什么这个机器人问题需要 MDP，而不只是 A* 或 Dijkstra?**
2. **为什么已经知道运动概率，仍然很难计算最优动作?**
3. **为什么你的系统需要 MCTS、风险感知启发式和 Monte Carlo Simulation 一起工作?**

我重新核对了你的 GitHub 当前代码、`A2/partB.tex` 报告和老师的 Week 5–6 课件。我们按照报告的 B1 → B2 → B3 → B4 顺序学习，每次只深入一部分。

先谈 Claude 所说的“理论上限”

Claude 的判断有合理的物理直觉：**你的任务中存在无法通过规划算法完全消除的运动随机性**。

例如，某架无人机即使选择了正确方向，也可能因为随机偏移撞上障碍物。对于一个必须经过狭窄通道的任务，这种风险可能无法完全避免。

但要严格区分：

- **不可避免的随机风险：**可以由运动模型支持。
- **当前算法已经达到理论最优成功率：**需要额外的数学证明。

设 π 是一个规划策略，任务成功率的理论最优值为：

$$\boxed{P_{\max} = \sup_{\pi} P_{\pi}(\text{所有无人机成功到达目标})}$$

要证明你的算法达到这个上限，需要计算或严格约束 $P_{\max}$，再与算法成功率比较。

目前的 13/20、3/10 等实验数据，不能单独证明已经达到理论上限。它们只能说明算法在这些随机环境中的实际表现。

另外还有一个特别重要的区别：**你的 MDP 优化目标是期望折扣奖励，不是单纯最大化成功率**。有时候更快但略有风险的策略，可能具有更高的期望奖励。

这个区别是理解整份作业的关键。

整个 Part B 的知识地图



图 1: Part B 知识地图

从智能机器人领域的角度，Part B 贯穿三个层次：

第一层：建模（Modelling）。把现实任务翻译成可以计算的数学问题。

第二层：决策（Planning）。在运动结果不确定的情况下，寻找期望回报较好的动作。

第三层：验证（Evaluation）。通过实际随机实验，判断算法是否真的有效。

今天先把第一层讲透。

1. 首先，机器人到底在解决什么？

想象你控制四架无人机。

每架无人机都有自己的终点，环境中存在障碍物。你需要同时决定四架无人机下一步怎么走。

如果运动完全确定，问题比较直观：找到一组不发生碰撞的路径，然后执行即可。

但老师给定了一个额外条件：

你下达的动作，不一定是无人机最终执行的运动方向。

例如，你命令无人机向右移动：



图 2: 预期移动与随机偏移

绿色为想执行的方向；红色为运动扰动后可能发生的其他移动。图仅用于说明随机动作效果。

这意味着规划器不能只回答：

> 从当前位置到目标，最短路径是哪一条？

而要回答：

> 我现在采取哪个动作，考虑随机运动、碰撞和后续决策后，长期期望收益最好？

这就是 **Sequential Decision Making under Uncertainty（不确定性下的序贯决策）**。

2. 为什么用 MDP？

MDP 是 Markov Decision Process，马尔可夫决策过程。

你的报告将它写成：

$$\boxed{\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \gamma)}$$

不要急着记五个符号。

它们实际上对应五个非常自然的问题：

数学对象	要回答的问题
S：State	机器人当前处于什么情况?
A：Action	机器人现在可以做什么?
T：Transition	做出动作后，可能发生什么？概率是多少?
R：Reward	发生这样的结果，好还是不好?
γ：Discount Factor	如何权衡现在与未来的收益?

MDP 的本质，就是用这五个对象，把随机环境下的连续决策过程数学化。

还有一个必须掌握的性质：

$$\boxed{P(s_{t+1} \mid s_{0:t}, a_{0:t}) = P(s_{t+1} \mid s_t, a_t)}$$

这叫 **Markov Property（马尔可夫性质）**。

意思是：如果当前状态定义得足够完整，那么在预测下一状态时，不需要重新查看全部历史。

现在我们检查你的 B1 状态是否满足这个要求。

3. State：为什么状态不仅包含坐标?

报告写的是：

$$\boxed{s_t = \left((p_i, r_i)_{i=1}^{N}, t\right)}$$

逐个解释。

p_i：第 i 架无人机的位置

$$p_i = (x_i, y_i, z_i)$$

例如，两个无人机分别位于：

$$p_1 = (2, 3, 1), \qquad p_2 = (5, 2, 0)$$

由于需要集中式规划，必须同时知道两个位置。

r_i：是否已经到达目标

$$r_i \in \{0, 1\}$$

- $r_i = 0$：还没有到达目标。
- $r_i = 1$：已经到达目标。

为什么需要这个变量?

因为老师提供的仿真器有一条非常重要的规则：

无人机到达目标后，会永久停留在那里，并继续占据那个格子。

看实际代码：

```
targets[reached] = prev_pos[reached]
```

这就是你之前共享走廊问题产生复杂协调需求的根本原因。

并且，无人机第一次到达目标时才获得目标奖励。因此 reached 同时影响未来运动和奖励计算。

t：当前时间步

为什么还需要 t?

因为老师规定了最大执行步数 H。

假设两次出现完全相同的无人机位置：

- 第一次只执行了 5 步，还有 75 步。
- 第二次已经执行了 79 步，只剩 1 步。

虽然位置相同，但未来能够做出的决策数量不同。

所以严格的有限时域 MDP 必须能够区分这两种情况。

在实际实现中：

```
remaining = max(
    0,
    int(self.cfg.max_num_steps) - self.steps
)
```

报告在数学状态中明确写了 t，而代码使用 self.steps 在规划器外部维护时间信息。

这里体现了一种重要的工程原则：数学状态的定义与程序内部的数据表示不必完全相同，但必须保留决策所需的信息。

为什么报告还定义了吸收状态 $\perp$?

报告写道：

$$T(\perp, a, \perp) = 1$$

它表示任务一旦终止，就不会再回到正常状态。

可能的终止原因包括成功、碰撞和达到时间限制。

在代码中，终止由 done 和真实执行的时间限制表示；报告用 $\perp$ 将它们统一为数学上的吸收状态。

4. Action：为什么是联合动作?

对于三维运动，每架无人机的位移为：

$$(\Delta x, \Delta y, \Delta z) \in \{-1, 0, 1\}^3$$

排除原地不动：

$$3^3 - 1 = 26$$

因此 N 架无人机的联合动作：

$$\boxed{a = (a_1, \dots, a_N)}$$

联合动作空间的大小是：

$$\boxed{|A| = 26^N}$$

四架时达到：

$$|A| = 456{,}976$$

这引出了整个 Part B 最重要的计算困难：

单架无人机的动作不多，但集中式联合决策会让动作空间随无人机数量指数增长。

而且，仅仅计算出四架无人机各自的最短路径还不够，因为它们的动作会相互影响。

对应代码

在 `planner.py`：

```
def encode_action(digits, base):
    value, multiplier = 0, 1
    for digit in digits:
        value += int(digit) * multiplier
        multiplier *= base
    return value
```

其数学形式为：

$$a_{\text{int}} = \sum_{i=1}^{N} a_i B^{i-1}$$

这只是联合动作的编码方式，不是规划算法本身。

5. Transition：这是 B1 的数学核心

现在来到最重要的部分。

你的报告写了：

$$\boxed{T(s, a, s') = \sum_{y : F(s,y) = s'} \prod_{i=1}^{N} q_i(y_i \mid p_i, r_i, a_i)}$$

看起来复杂，但它实际上只有两步。

第一步：分别产生每架无人机的随机移动

仿真器中：

```
d = self.dist_field[
    prev_pos[:, 0],
    prev_pos[:, 1],
```

```
    prev_pos[:, 2]
]

p_succ = d / (d + self.cfg.alpha)
```

即：

$$p_{\text{succ}} = \frac{d}{d + \alpha}$$

其中 d 是当前位置的障碍物距离场数值，α 控制运动的不确定程度。

假设：

$$d = 2, \qquad \alpha = 0.5$$

那么：

$$p_{\text{succ}} = \frac{2}{2.5} = 0.8$$

在所有方向都不越界的情况下，预期动作被赋予 0.8 的概率，其余方向分享剩下的 0.2。

注意，代码还会排除越界方向并重新归一化，因此靠近边界时最终概率可能不同。

对于第 i 架无人机，这个概率分布记作：

$$q_i(y_i \mid p_i, r_i, a_i)$$

报告中的乘积：

$$\prod_{i=1}^{N} q_i(y_i \mid p_i, r_i, a_i)$$

表示给定当前状态和联合动作，各无人机的原始运动结果按照模型条件独立采样。

第二步：统一处理碰撞和终止

这一步用函数 $F(s, y)$ 表示。

仿真器会检查：

```
obstacles_hit = self.obstacles[
    targets[:, 0],
    targets[:, 1],
    targets[:, 2]
]
```

然后检查无人机之间的碰撞：

```
_, counts = np.unique(
    next_pos,
    axis=0,
    return_counts=True
```

```
)

veh_collisions = int(counts[counts > 1].sum())
```

这里有一个非常深刻的区别：

每架无人机的随机运动可以条件独立，但联合任务的结果并不独立。

例如，两架无人机分别以很高概率到达同一个格子。

从各自运动角度看，两次移动都可能执行成功；但从联合任务角度看，它们发生了碰撞。

因此必须在独立采样之后，再统一判断整个系统的下一状态和任务后果。

这就是 B1 转移公式中：

$$\underbrace{\prod_i q_i}_{\text{各无人机随机运动}} \longrightarrow \underbrace{F(s,y)}_{\text{全局碰撞与状态更新}}$$

的真正含义。

它也回答了一个问题：**为什么这里必须是 Centralised Planning，而不是把几个单无人机规划器简单并排运行？**

因为联合动作的价值受到其他无人机的状态和行动影响。

6. Reward：规划器究竟在优化什么？

这是另一个特别重要的知识点。

你的报告给出的采样奖励是：

$$\boxed{\tilde{r} = Nc_{\text{step}} + C(K_O + K_V) + G\sum_i \Delta r_i}$$

可以理解为三部分：

Nc：**运动时间成本**。每执行一步都付出负奖励，鼓励尽快完成任务。

$C(K_O + K_V)$：**碰撞惩罚**。撞到障碍物或其他无人机，产生较大的负奖励。

$G\sum_i \Delta r_i$：**首次抵达目标的奖励**。鼓励每架无人机完成自己的任务。

例如，在一个两架无人机、步成本为 -1、碰撞惩罚为 -100、目标奖励为 100 的配置中：

实际发生的事件	本步奖励
两架都还在正常移动	-2
一架首次到达目标，无碰撞	98
一架撞到障碍物，无新目标到达	-102

这里的奖励是**一次随机执行后实际得到的数值**：

$$\tilde{r}(s,a,y)$$

而数学上的期望奖励是：

$$R(s,a) = \mathbb{E}[\tilde{r}(s,a,y) \mid s,a]$$

区别很重要：前者是一次模拟结果，后者要对所有可能结果求概率加权平均。

MCTS 正是通过许多次随机模拟，逐渐估计动作的长期价值。

7. 最终目标：不是走最短路，而是最大化期望累计奖励

报告给出：

$$\boxed{\max_{\mu} \mathbb{E}_{\mu}\left[\sum_{t=0}^{\tau-1} \gamma^t \tilde{r}_t\right]}$$

这里：

- μ：决策策略，规定每个状态应该选择什么动作。
- γ：折扣因子。
- τ：任务实际终止时刻。
- $\tilde{r}_t$：第 t 步获得的奖励。

从研究生层面的理解来看，重点在于：

一条路径的价值，不只是几何长度，而是执行这条策略时所有可能随机结果的期望累计收益。

例如，一条路径很短，但会多次贴着障碍物行进；另一条稍长，但大部分时间远离障碍物。MDP 会通过转移概率、碰撞惩罚和未来奖励综合权衡这两种选择。

这也是为什么后面 B2 的规划器需要估计：

$$Q(s,a)$$

即**当前状态下选择动作 a，随后继续决策，能够获得多大期望累计回报。**

MCTS 要解决的问题，正是如何在有限计算预算下近似比较这些 $Q(s,a)$。

第一讲总结：你要建立的思维模型

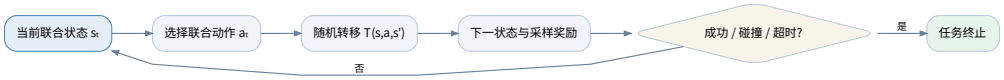


图 3: MDP 闭环流程

你现在需要把三件事牢牢记住：

第一，MDP 是问题的数学模型，不是具体的求解算法。

第二，运动随机性由 T 描述，目标由 R 描述。它们决定了什么策略是好的。

第三，已知 T 和 R，不代表最优策略已经知道。因为要考虑未来许多步、多个无人机及所有可能的随机后果，求解仍然很困难。

对应到程序，最关键的一行是：

```
successor, reward, done, info = execution.step(action)
```

它就是整个 MDP 一次真实状态转移的工程实现。

而 `planner.py` 中的：

```
return self.env.simulate(state, action)
```

则允许规划器在真正行动之前，提前抽样研究不同动作可能产生的未来。

这两行代码之间的区别，正是下一讲 B2：Online MDP Solving 与 MCTS 的起点。

一个检验你是否真正理解 B1 的问题

假设有两架无人机，每架已经分别找到了自己的最短路径，并且两条路径在几何上没有共享格子。

为什么在这个作业中，我们仍然不能保证它们不会发生碰撞?

关键不在路径规划本身，而在你刚学到的转移模型 T。如果能解释清楚这个问题，就说明你已经抓住了 B1 的核心。

下一讲我们将从 **Bellman Equation → 为什么精确求解困难 → MCTS 如何近似求解 → `_search()` 逐行分析** 这条主线进入 B2。

第二讲：从 Bellman Equation 到 MCTS

对应 Week 5《MDP Solving》、Week 6《MDP Solving Continued》、A2 Part B Q2，以及你 GitHub 中的 `planner.py`。

第一讲我们建立了 MDP：

$$\mathcal{M} = (S, A, T, R, \gamma)$$

现在我们知道了无人机的状态、可执行的动作、运动发生随机偏移的概率，以及完成任务和发生碰撞的奖励。

但是，这些知识还没有告诉我们：

在当前状态下，究竟应该选择哪一个联合动作?

这就是第二讲要解决的核心问题。

我希望你理解一个贯穿本讲的思想：

> **MDP 告诉我们什么是好的决策；Bellman 方程刻画最优决策；MCTS 则试图在有限计算时间内近似找到这样的决策。**

一、Bellman 方程：为什么不能只考虑下一步?

先从一个简单问题开始。

假设无人机有两种选择：

- 动作 A：下一步就靠近目标，但未来必须经过障碍物密集区域。
- 动作 B：下一步看起来绕远了，但之后可以通过更安全的区域。

如果使用纯粹的贪心策略（Greedy），可能会选择 A。

但我们真正关心的并不是下一步距离减少了多少，而是**整个任务的期望累计奖励**。

1. 定义状态价值 $V(s)$

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\tau-1} \gamma^k r_{t+k} \,\middle|\, s_t = s\right]$$

它表示：

从状态 s 出发，按照策略 π 不断采取动作，未来预计能够获得多少累计折扣奖励。

注意，$V(s)$ 衡量的是**状态的价值**。

2. 定义动作价值 $Q(s, a)$

$$Q^{\pi}(s, a) = \mathbb{E}\left[r_t + \gamma V^{\pi}(s_{t+1}) \mid s_t = s, a_t = a\right]$$

它表示：

在状态 s 下，首先执行动作 a，然后按照策略 π 继续行动，预计能获得多少奖励。

比较两个公式：

$$\underbrace{V(s)}_{\text{这个状态有多好}} \qquad \underbrace{Q(s, a)}_{\text{现在选择这个动作有多好}}$$

你的 MCTS 主要就是在估计不同候选动作的 $Q(s, a)$。

3. Bellman Optimality Equation

对于还剩 h 步的有限时域问题：

$$\boxed{Q_h^*(s,a) = \mathbb{E}\left[r + \gamma V_{h-1}^*(s') \mid s, a\right]}$$

$$\boxed{V_h^*(s) = \max_{a \in A(s)} Q_h^*(s,a)}$$

其中终止状态的未来价值为零，$V_0^*(s) = 0$。

这两个公式合起来就是最优性递推。

理解第一条公式：

$$\underbrace{Q_h^*(s,a)}_{\text{选择动作}a\text{的长期价值}} = \underbrace{\mathbb{E}[r]}_{\text{当前奖励}} + \underbrace{\gamma \mathbb{E}[V_{h-1}^*(s')]}_{\text{未来价值}}$$

例如，当前选择一个动作，它可能带来：

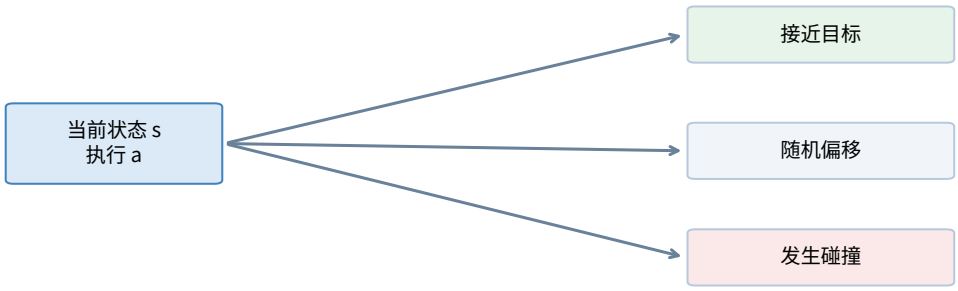


图 4: 同一动作的随机结果

同一个动作可能产生多个随机结果，各自具有不同的奖励及后续价值。

因此你必须把这些可能结果按照它们的概率综合考虑。

这就是 Bellman 方程中的期望算子 $\mathbb{E}$。

这里有一个研究生阶段非常重要的认识：

最优动作不一定具有最高的立即奖励，也不一定使几何距离下降最多，而是具有最高的期望长期价值。

你的报告中，优化目标也是期望折扣回报，而非直接最大化成功率。

二、Bellman 方程已经有了，为什么还需要 MCTS?

这可能是整份 Part B 最值得理解的问题。

如果我们已经知道最优性方程，似乎只需要把它计算出来就行了。

问题在于：**计算规模可能过于庞大。**

1. 动作数量增长

对于四架三维无人机：

$$|A| = 26^4 = 456{,}976$$

一个状态就可能对应几十万个联合动作。

2. 每个动作还对应多个随机后继

例如，某个联合动作 a：

$$(s, a) \longrightarrow \begin{cases} s'_1 \\ s'_2 \\ s'_3 \\ \vdots \end{cases}$$

这些后继来自运动扰动的不同组合。

所以求解一个 $Q(s, a)$，需要考虑多个可能的 s'。

而每个 s' 又需要考虑下一步动作。

再往后，又会有新的随机后继。

这种嵌套递推使完整求解的计算量很大。

3. Offline 与 Online Planning 的本质区别

Week 5 第 20 页讨论了 Computing the Policy Online。

Offline Planning	Online Planning
尝试预先计算大量状态的策略或价值	集中计算当前真实状态下的动作
更强调提前求得完整策略	计算与实际执行交替进行
可能需要处理大量无关状态	重点搜索当前状态附近的未来
计算规模大时比较困难	可以在有限预算后给出近似决策

对于你的作业，正确的工作流程是：

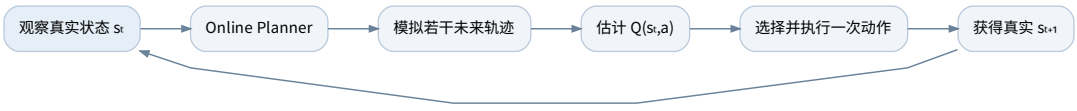


图 5: Online Planner 与真实执行交替

这里最关键的区别是：

规划器可以模拟很多次未来，但真实环境每次只执行一个动作。

对应你的代码：

```
# 在规划器内部进行假想的未来模拟
successor, reward, done, _ = self.env.simulate(state, action)

# 真正让无人机执行选定的动作
successor, reward, done, info = execution.step(action)
```

前者在 `planner.py`；后者在 `run_u8076655.py`。

这就是为什么 MCTS 特别适合老师提供的 `MultiDroneUnc`：它直接提供了一个能够采样随机后继的 **Generative Model（生成式模型）**。

你不必提前构建完整的 $T(s, a, s')$ 矩阵。

三、MCTS 的本质：通过模拟，学习哪些动作值得继续探索

MCTS 全称：

Monte Carlo Tree Search（蒙特卡洛树搜索）

它结合两个思想。

Monte Carlo：用随机采样来估计难以精确计算的期望。

Tree Search：将计算集中在目前看来有价值的决策分支上。

因此 MCTS 可以理解为：

通过有限次随机模拟，逐渐改进当前动作的价值估计

注意，MCTS 不是强化学习中的 DQN，也不需要训练神经网络。

它是在当前决策时刻使用模型进行规划。

为什么是树?

假设无人机当前处于状态 s_0，有两个候选动作 a_1, a_2。

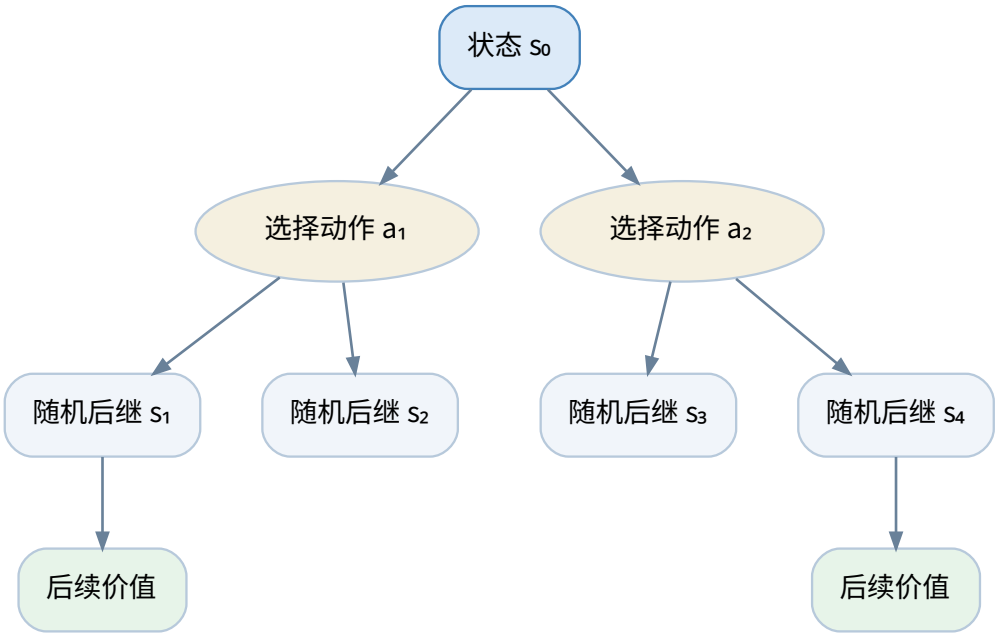


图 6: 带有动作与随机后继的搜索树

必须注意，这里有两种不同的分支：

动作分支：规划器可以选择哪个动作？

随机结果分支：某个动作执行后，环境实际产生哪个后继？

Week 5 第 31 页将 MDP 的 MCTS 搜索树描述为 AND–OR Tree。

它表达的就是这种“选择动作”与“考虑随机结果”的交替结构。

你代码中的：

```
edge.children
```

正是用来保存同一个动作对应的不同随机后继状态。

MCTS 的四阶段

Week 5 第 27–30 页介绍了四个组成部分：

阶段	解决的问题
Selection	已有候选动作中，继续模拟哪个？
Expansion	什么时候允许新动作或新状态进入树？
Simulation / Rollout	搜索树尚未展开的未来，怎么估计？
Backpropagation	模拟得到的回报，怎样更新之前的决策价值？

接下来我们真正进入你的 planner.py，看这四个阶段如何组成一个可以运行的在线求解器。

四、第一阶段：Selection —怎么决定继续探索哪个动作?

先看你的代码:

```
unseen = [
    a for a, edge in node.actions.items()
    if edge.visits == 0
]

if unseen:
    action = unseen[0]
else:
    log_n = math.log(node.visits + 1)
    action = max(
        node.actions,
        key=lambda a:
            node.actions[a].value
            + self.exploration *
              math.sqrt(log_n / node.actions[a].visits)
    )
```

对应 planner.py 中的 _search()。

这段代码分成两种情况。

情况一：存在还没有模拟过的候选动作。

优先模拟它一次。

因为如果一个动作从未被尝试，我们根本没有它的价值估计。

情况二：已有候选动作都至少模拟了一次。

使用 UCT:

$$a^* = \arg\max_a \left[Q(s,a) + c_{\text{UCT}} \sqrt{\frac{\ln(N(s)+1)}{N(s,a)}} \right]$$

当前代码:

$$c_{\text{UCT}} = 0.35$$

这就是 Week 5 第 33 页讲的 UCB/UCT。

为什么不直接选择 Q 最大的动作?

假设有两个动作:

动作	当前平均回报 Q	模拟次数
A	0.18	100
B	0.14	10

只看平均回报，会选择 A。

但是 B 只模拟了 10 次，我们可能尚未充分了解它。

假设当前节点总访问次数为 110，那么：

$$\mathrm{UCT}(A) = 0.18 + 0.35\sqrt{\frac{\ln 111}{100}} \approx 0.256$$

$$\mathrm{UCT}(B) = 0.14 + 0.35\sqrt{\frac{\ln 111}{10}} \approx 0.380$$

所以 MCTS 此时反而会选择 B。

原因是：

B 的平均收益虽然暂时较低，但它被探索得还不充分。

这叫 Exploration–Exploitation Trade-off。

交互理解 UCT 探索系数（PDF 静态版）

探索系数 c	UCT(A)	UCT(B)	此时优先探索
0	0.180	0.140	A
0.20	0.223	0.277	B
0.35	0.256	0.380	B
0.80	0.354	0.689	B

原对话中的可拖动 UCT 探索系数滑块在 PDF 中替换为同一模型的静态数值表。这些数字是教学示例，不是代码的真实运行数据。

注意：UCT 是搜索时使用的动作选择公式，并不等于最终真实执行时的动作选择规则。

你的代码在搜索完成后，还有一个独立的 Root Decision 机制。后面会讲。

五、第二阶段：Expansion —为什么不能展开所有联合动作?

这里是你实现与最基本 MCTS 的一个重要区别。

理论上，四架三维无人机最多有：

$$26^4 = 456{,}976$$

个联合动作。

假设把所有动作都加入搜索树，然后对每个动作平均分配模拟次数。

在单步最多 2,000 次搜索试验的预算下，连每个动作模拟一次都做不到。

因此必须回答：

哪些动作值得进入搜索树？应该允许多少个动作进入?

你的方法叫 **Progressive Widening（渐进拓宽）**。

1. 核心公式

代码：

```
width = max(
    1,
    math.ceil(
        self.widening * math.sqrt(node.visits + 1)
    )
)
```

当前：

$$\texttt{widening} = 0.5$$

因此：

$$\boxed{K(s) = \lceil 0.5\sqrt{N(s)+1} \rceil}$$

同时还受可提案动作数量的限制。

我们来看几个数：

节点访问次数 $N(s)$	最多允许的候选动作数
0	1
100	6
500	12
2,000	23

发现没有？

即使原始动作空间中有几十万个动作，搜索树在访问 2,000 次的量级下，也只允许加入几十个候选动作。

这就大大提高了有限计算预算的利用效率。

但它也引入一个代价：**如果真正好的动作没有被加入候选集合，UCT 根本无从发现它**。

2. 那么谁决定加入哪些动作？

对应代码：

```
action = self._proposal(
    state,
    randomize=bool(node.actions)
)
```

`_proposal()` 是你算法里非常关键的启发式动作生成函数。

它不是纯随机生成联合动作，而是使用：

Risk-aware Dijkstra + 目标距离 + 无人机占用协调

具体而言，它考虑目标剩余距离、障碍物附近的运动偏移风险，以及其他无人机占用目标格子的风险。

第一次加入动作时，使用 Greedy Proposal；之后在有偏随机采样和探索采样之间选择。

所以你的 Expansion 实际上有两层：

$$\underbrace{\text{Progressive Widening}}_{\text{决定加入多少动作}} + \underbrace{\text{Heuristic Proposal}}_{\text{决定优先加入哪些动作}}$$

这是整套系统最重要的工程设计之一。

这里要区分课程依据：

Week 5 明确讲了 MCTS Expansion；Progressive Widening 和这套风险感知动作提案，是你针对本题进行的具体设计扩展，而不是课件直接给定的实现。

六、第三阶段：Simulation —为什么还需要 Rollout?

这是我希望你特别吃透的部分。

MCTS 的搜索树不可能无限往未来展开。

假设当前无人机距离终点还要 20 步，而你的搜索树只探索了 5 步。

那么，搜索到第 5 步时，怎么判断这个状态好不好?

一种极端做法是：

$$V(s) = 0$$

也就是把尚未模拟的未来价值视为零。

但这可能产生严重偏差。

因为一个已经非常接近目标的状态，和一个仍然远离目标的状态，显然不应该具有相同价值。

1. Rollout 的作用

Rollout 的思想是：

从新发现的状态出发，再按照一个比较简单的策略模拟未来几步，以此估计状态价值。

Week 5 第 34 页明确建议可以使用确定性问题的 Greedy 解作为启发式 Rollout。

你的代码：

```
for _ in range(min(self.rollout_depth, remaining)):
    explore = self.rng.random() < self.rollout_epsilon

    action = self._proposal(
        state,
        randomize=False,
        explore=explore
    )

    state, reward, done, _ = self._simulate(
        state, action
    )
```

默认参数：

$$H_{\text{rollout}} = 8$$

$$\epsilon_{\text{rollout}} = 0.10$$

表示 Rollout 最多额外模拟 8 步，每一步以 10% 的概率使用探索提案，其余情况使用目标导向启发式。

但这 10% 是**规划策略的探索概率**，与真实无人机的运动扰动概率不是一回事。

2. 为什么还需要 Leaf Value?

如果 Rollout 的 8 步走完了，仍然没有抵达目标怎么办?

你的代码:

```
tail = (
    0.
    if terminal
    else self._leaf(state, remaining - len(rewards))
)
```

即:

- 已经终止：未来价值为 0。
- 还没有终止：调用 `_leaf()` 估计剩余价值。

你的 Leaf Value 主要利用 **BFS 最短剩余步数**，估计还需要支付多少步成本、未来可能拿到多少目标奖励。

注意这个分工:

组件	使用什么信息
`_cost_map()`	风险加权 Dijkstra，用于指导动作
`_distance_map()`	BFS 最短距离
`_leaf()`	基于 BFS 距离估计未模拟完的价值

因此，风险感知 Dijkstra 并没有直接取代 MDP 价值函数。

它主要让搜索更容易找到好的方向。

Leaf Value 则是在搜索视野结束时提供近似估值。

3. Rollout 到底在估计什么?

假设它模拟三步，得到奖励:

$$r_0, r_1, r_2$$

最终的近似回报是:

$$\boxed{\widehat{G} = \frac{r_0}{\kappa} + \gamma \frac{r_1}{\kappa} + \gamma^2 \frac{r_2}{\kappa} + \gamma^3 \frac{\widehat{V}(s_3)}{\kappa}}$$

其中 κ 是奖励归一化系数。

代码中 Rollout 使用了倒序折扣累加:

```
for reward in reversed(rewards):
    tail = reward + self.gamma * tail
```

注意，此处 rewards 已经除以 κ，而 _leaf() 返回值也已经归一化。

为什么说 Rollout 可能影响算法性能?

因为我们真正想要的是最优价值:

$$V^*(s)$$

但 Rollout 通常只是按照一个启发式策略执行，所以它给出的只是近似价值:

$$\widehat{V}(s)$$

如果启发式策略本身对某些情况判断错误，那么 MCTS 也可能继承这种偏差。

这正是你墙壁缺口场景中出现反复绕行的可能原因之一，但仅凭轨迹不能断定它是唯一原因。

七、随机结果：为什么同一个动作还会有多个子节点?

看 _search() 里的代码:

```
successor, reward, done, _ = self._simulate(
    state, action
)

key = np.asarray(
    successor, dtype=np.int32
).tobytes()

child = edge.children.get(key)
```

这段代码实现了一个与普通确定性树搜索不同的机制。

假设在状态 s 下选择动作 a。

第一次模拟:

$$s \xrightarrow{a} s'_1$$

第二次模拟:

$$s \xrightarrow{a} s'_2$$

两个随机后继可能不同。

代码以实际后继状态为键，将它们保存为同一个动作下的不同子节点。

如果之前没有见过这个后继:

```
if child is None:
    child = Node()
    edge.children[key] = child
    tail = self._rollout(successor, remaining - 1)
```

新建子节点，并通过 Rollout 估值。

如果之前已经见过：

```
else:
    tail = self._search(
        child, successor, remaining - 1, depth + 1
    )
```

递归进入已有搜索树，继续探索。

你应该从这里建立一个直觉：

MCTS 不是假设无人机一定执行成功，而是在一棵不断增长的树中积累不同随机结果的信息。

这也是为什么它能处理运动不确定性。

八、第四阶段：Backpropagation —模拟结果怎样更新 Q?

现在一条模拟轨迹已经结束，我们得到一个回报估计。

接下来要把这次得到的信息反馈给搜索树。

你的 `_search()`：

```
value = float(reward) / self.scale + self.gamma * tail

edge.add(value)
node.visits += 1

return value
```

最核心的数学公式是：

$$\boxed{G = \frac{r}{\kappa} + \gamma G_{\text{future}}}$$

假设某个动作经过三次模拟，得到归一化回报：

$$0.2, \quad 0.4, \quad 0.1$$

其平均价值为：

$$Q(s,a) = \frac{0.2 + 0.4 + 0.1}{3} \approx 0.233$$

第 4 次模拟得到：

$$G_4 = 0.5$$

那么新的价值：

$$Q_{\text{new}} = \frac{3(0.2333) + 0.5}{4} = 0.3$$

实际上不需要保存之前的所有回报。

可以增量更新：

$$\boxed{Q_{\text{new}} = Q_{\text{old}} + \frac{G - Q_{\text{old}}}{n_{\text{new}}}}$$

你的 `Edge.add()` 实现的正是这个公式。

另外，它还使用 **Welford's Recurrence** 更新回报的离差平方和，供最后的根节点动作切换使用。

这里有一个重要概念区别：

这不是 Q-learning。

Q-learning 的典型更新需要通过估计的下一状态最优动作价值来构造 TD Target；这里的 Q 值主要来自树搜索和 Rollout 产生的 Monte Carlo 回报样本。

九、整套 MCTS 在 _search() 中怎样串起来?

现在你已经理解四个阶段，可以直接阅读简化后的整体逻辑。

```
def search(node, state, remaining, depth):
    if terminal_or_horizon_reached(state, remaining):
        return 0.0

    if depth >= TREE_DEPTH:
        return rollout(state, remaining)

    # 1. Expansion: admit new actions when allowed
    maybe_add_guided_action(node, state)

    # 2. Selection: select an admitted action
    action = select_by_unvisited_or_UCT(node)

    # 3. Sample one stochastic successor
    next_state, reward, done = simulate(state, action)

    if done:
        future = 0.0
    elif successor_is_new(node, action, next_state):
        create_successor_node(...)
        future = rollout(next_state, remaining - 1)
    else:
        future = search(
            child_node(...),
            next_state,
            remaining - 1,
            depth + 1
        )

    # 4. Backpropagation
    value = reward / SCALE + GAMMA * future
    update_action_mean_and_variance(node, action, value)
    node.visits += 1

    return value
```

以上是教学用伪代码，省略了真实代码中的时间截止检查、动作唯一性检查等工程细节，并非可以直接替换的

函数。

你可以把一次搜索理解为：

$$\boxed{\text{选择一个动作} \rightarrow \text{采样一个随机未来} \rightarrow \text{估计后续价值} \rightarrow \text{更新当前动作评价}}$$

这就是整个 MCTS 的核心循环。

十、最容易忽视的地方：搜索结束后，为什么不直接执行 Q 最大的动作？

这是你当前实现比较特别的部分，也是我认为需要真正理解的地方。

通常介绍 MCTS 时，最后可以选择访问次数最多或估计价值最高的根节点动作。

但你的代码采用了：

Anchored Root Decision（以贪心策略为锚点的根节点决策）。

原理如下。

假设：

- a_g：风险感知 Greedy 推荐的动作。
- a：MCTS 搜索发现的另一个动作。

你不希望仅仅因为 $Q(s,a)$ 比 $Q(s,a_g)$ 高一点，就轻易放弃已有的强启发式策略。

因为 Monte Carlo 估计存在随机波动。

因此代码定义：

$$\boxed{m(a) = Q(a) - Q(a_g) - z\hat{\sigma}\sqrt{\frac{1}{N(a)} + \frac{1}{N(a_g)}}}$$

其中：

$$z = 1$$

只有当：

$$m(a) > 0$$

时，才考虑切换到其他动作，并在正裕量中选择最大的那个。

如果没有这样的动作，就继续执行 Greedy。

它的设计思想是：

Greedy 提供可靠基线；MCTS 负责发现足以证明自己更有价值的替代动作。

但是这里的方差阈值只是一种启发式保护机制。MCTS 样本受自适应选择影响，所以不能把它直接视为严格的统计显著性检验或最优性证明。

实际代码还有两个后备条件：如果 Greedy 没被搜索访问过，按访问次数优先选择已访问动作；如果没有完成任何搜索试验，则直接返回 Greedy。

这是一个很好的研究启发：

采样预算有限时，合理的决策规则有时与搜索本身同样重要。

在你 B4 的消融中，切换回 Most-visited 根节点规则会使惩罚完成步数平均增加约 3.37 步，这为当前 Anchored Root Decision 提供了实证支持。

十一、最后：Online Planner 怎么控制时间?

对应 `plan()`:

```
fallback = self._proposal(state, randomize=False)
root = Node()

while (
    completed < self.max_simulations
    and remaining > 0
    and time.perf_counter() < self.deadline
):
    self._search(root, state, remaining, 0)
    completed += 1
```

当前设置:

$$K_{\max} = 2000, \qquad \text{默认规划预算} = 9.5\text{秒}$$

停止条件是：模拟试验次数达到上限，或者接近时间截止点。

这里要特别注意:

2,000 次 MCTS Trial 不等于调用 `simulate()` 2,000 次。

一次 MCTS Trial 里面可能包含多次树内状态转移，以及多步 Rollout。

因此:

$$\text{Generative Model Calls} \geq \text{Completed MCTS Trials}$$

通常会明显更多。

最后，真实环境执行一个联合动作，观察下一状态，然后**重新构建根搜索树**进行下一步规划。

这就是 Replanning，也体现了在线规划的闭环特征。

第二讲总结

我们现在可以从高层次重新理解你的算法:

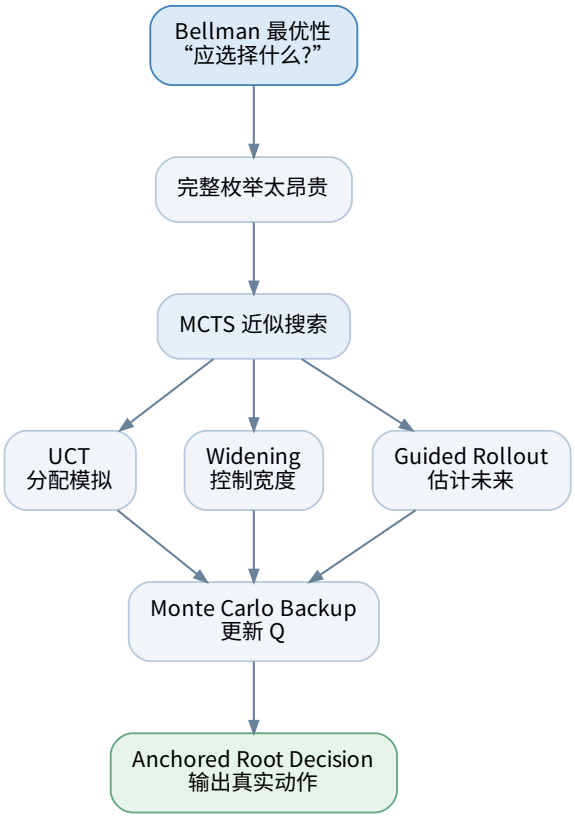


图 7: Bellman 到 MCTS 的层次关系

从课程知识体系看:

层次	课程概念	你代码中的具体实现
数学目标	Bellman Optimality	最大化期望折扣回报
近似方法	Monte Carlo Tree Search	`_search()`
搜索选择	UCT	`_search()` 中的 UCB
分支控制	MCTS Expansion	Progressive Widening
未展开的未来	Heuristic Rollout	`_rollout()`、`_leaf()`
价值估计	Monte Carlo Backup	`Edge.add()`
最终决策	当前状态下的动作选择	`_anchored()`、`plan()`

如果只允许你用一句话解释当前算法，我建议这样理解:

我们使用风险感知启发式提出有希望的联合动作，再利用 MCTS 对它们的随机未来进行有限次数的模拟，通过 Monte Carlo 回报估计长期价值，最后谨慎地选择一个动作真实执行。

这里还有一道检验题:

假设 MCTS 中某个动作已经被模拟 500 次，平均 $Q = 0.25$；另一个动作只模拟了 5 次，平均 $Q = 0.20$。

为什么搜索阶段仍然可能优先选择第二个动作，但真实执行阶段未必选择它？

这个问题能够同时检查你是否理解 UCT 的探索机制，以及 Anchored Root Decision 与 Selection 的区别。

第三讲我们将专门深入你的算法最有工程特色的部分：`_proposal()`、BFS、Risk-aware Dijkstra 与 Rollout。重点解释为什么它们不是另一套 MDP Solver，而是让 MCTS 在有限时间内变得实用的关键。

第三讲：B2 的工程灵魂——BFS、Risk-aware Dijkstra 与 _proposal()

本讲是新增内容。依据最新报告 Question 2、Appendix Algorithm Definitions，以及 `planner.py` 的 `_slip_model()`、`_distance_map()`、`_cost_map()`、`_options()`、`_slip_penalty()`、`_proposal()`。

前两讲解决了两件事：首先，为什么机器人随机导航是 MDP；其次，为什么 MCTS 可以在不知道完整价值函数的情况下，用有限次采样评估候选动作。

现在进入最有工程辨识度的问题：**一个行动空间最多有 456,976 个候选动作的规划器，要怎样在很短时间内把注意力集中到合理的动作上?**

答案不是“让 MCTS 变聪明”这么抽象。它必须拆成可以实现、可以验证的机制。

一、先建立三个不同的“距离”概念

在导航任务中，你至少会遇到三种概念。它们经常被混为一谈，但在这个作业中必须严格区分。

概念	典型形式	是否考虑静态障碍	是否精确考虑随机转移	在代码中的位置
几何直线距离	$\|p - g\|_2$	否	否	直觉参考
BFS 物理步数	$\delta_i(p)$	是	否	`_distance_map()`、`_leaf()`
风险加权路径成本	$J_i(p)$	是	近似风险	`_cost_map()`、`_proposal()`

注意最后一列的逻辑：**这些都是启发式信息**。真正的随机 MDP 最优价值 $V^*(s)$ 还需要考虑奖励、动作转移、其他无人机、时间限制与终止规则。

1. BFS 解决什么?

先假定动作完全执行成功，不考虑风扰动。地图中某些格子不可进入，要求从当前格子 p 到目标 g_i 的最少步数。

把每个可通行格子视为图节点, 允许的物理移动方向构成边。由于每条边都计为一步, 可以使用 **Breadth-First Search（BFS）**。

$$\boxed{\delta_i(p) = \min_{\pi: p \rightsquigarrow g_i} |\pi|}$$

如果根本不存在路径，设 $\delta_i(p) = \infty$。代码使用整数数组中的 -1 表示未到达或不可达，再在调用处换算成 ∞。

```
def _distance_map(self, goal, free):
    distance = np.full(self.shape, -1, dtype=np.int32)
    queue = deque([goal])
    distance[goal] = 0
    while queue:
        p = queue.popleft()
        # visit accessible neighbours and set d + 1
```

为何从目标反向运行 BFS，而不是每次从无人机当前位置跑到终点? 因为**地图和目标在整个任务期间不变**，可以预计算每个格子到目标的距离，再在每个决策步查询。空间换时间，这是在线规划的常用工程手段。

2. BFS 的局限

最短步数并不一定意味着最安全。

如果路线 A 在墙边经过 6 步，路线 B 绕远经过 8 步，BFS 只比较 6 与 8，却不知道贴墙状态的运动偏移风险可能更大。

因此需要一个新的成本：不仅为“走一步”付费，还为“所处位置附近的随机碰撞暴露”付费。

二、为什么能够从已知运动模型估计风险?

你的仿真器不是一个不可解释的黑箱。它的运动规律已知:

$$p_{\text{succ}}(p) = \frac{d(p)}{d(p) + \alpha}.$$

当前位置 p 附近可能存在 B 个物理移动方向；其中越界方向在真实转移中被置零，然后把剩余权重归一化。

报告 Appendix 定义:

$$\begin{aligned}
\pi(p) &= \frac{d(p)}{d(p) + \alpha}, \\
M(p) &= \pi(p) + (V(p) - 1)\frac{1 - \pi(p)}{B - 1}, \\
\ell(p) &= \frac{\pi(p)}{M(p)}, \\
\sigma(p) &= \frac{1 - \pi(p)}{(B - 1)M(p)}.
\end{aligned}$$

这里 $V(p)$ 是当前位置**留在网格内部**的物理邻居数量。在“预期移动方向合法”的情形，$\ell(p)$ 是预期方向的归一化执行概率，$\sigma(p)$ 是每个其他合法方向的归一化概率。

一个微妙但重要的认识：当预期目标合法且非障碍物时，在同一个当前位置 p，单步因偏移撞障碍物的概率，主要由当前位置附近有多少障碍方向决定，而不是由你想朝左还是朝右决定。选动作的核心因此是**改变未来经过哪些位置**，减少风险暴露，而不是幻想能在同一个位置通过选择方向完全消除随机风险。

1. 局部风险 $\rho(p)$

你代码中:

```
success = self.clearance / np.maximum(
    self.clearance + alpha, 1e-12
)
slip = (1 - success) / max(
    1, len(self.physical_vectors) - 1
)
valid = self._neighbour_sum(np.ones(self.shape, dtype=bool))
mass = np.maximum(success + (valid - 1) * slip, 1e-12)
self.land_intended = success / mass
self.land_slip = slip / mass
self.departure_risk = (
    self._neighbour_sum(~free) * self.land_slip
)
```

`_neighbour_sum(~free)` 统计周围物理邻居中有多少障碍格子，而 `land_slip` 给出每个替代方向的权重。

因此：

$$\boxed{\rho(p) = |\mathcal{N}(p) \cap O|\,\sigma(p)}$$

它估计**从当前位置执行合法命令时，因随机偏移撞上静态障碍物的单步概率**。这不是整条路线的精确碰撞概率，也不精确表达所有无人机之间的联合碰撞风险。

2. 风险加权路径成本 $J_i(p)$

你使用的成本大致是：

$$\boxed{c_{\text{guide}}(p) = 1 + \lambda\rho(p)}$$

这里的 1 是基本步数，风险惩罚 $\lambda\rho(p)$ 把局部碰撞概率转换成额外的启发式代价。

代码中风险权重由目标奖励、碰撞惩罚、每步成本等量级构造：

$$\lambda = \frac{w(G + |C|)}{\max(|c_{\text{step}}|,\, 10^{-3}(G + |C|),\, 10^{-9})}.$$

在典型参数 $w = 1, G = 100, C = -100, c_{\text{step}} = -1$ 下，$\lambda = 200$。

假设位置 P 的 $\rho(P) = 0.01$，位置 Q 的 $\rho(Q) = 0.05$，那么：

$$c_{\text{guide}}(P) = 3, \qquad c_{\text{guide}}(Q) = 11.$$

这表达的是搜索偏好，并不是说真实环境会在 P、Q 分别额外给出 -3 和 -11 的奖励。

原始 MDP 奖励仍由老师的 `simulate()` 返回。启发式代价只作用于搜索引导。

3. 为什么这里使用 Dijkstra 而不是 BFS?

BFS 要求每条边的成本相同。在这里，靠近障碍物的格子具有不同的风险成本，因此使用带权最短路算法 **Dijkstra**：

$$\boxed{J_i(p) = \min_{\pi: p \rightsquigarrow g_i} \sum_{q \in \pi} c_{\text{guide}}(q)}$$

代码从目标开始运行反向的 Dijkstra，将对应的未来离开位置成本累积到前驱格子。`_cost_map()` 在 `risk_scale == 0` 时回退为物理 BFS 距离。

```
if self.risk_scale == 0:
    return np.where(steps >= 0, steps.astype(float), math.inf)

enter = (1. + self.risk_scale * self.departure_risk).tolist()
heap = [(0., goal)]
# Repeatedly relax adjacent free predecessors.
```

一个需要牢牢记住的边界

Dijkstra 计算的是**近似确定性图上的成本**。它不是通过 Bellman Backup 得到的真实随机 MDP 最优价值函数。你把 Dijkstra 当作 MCTS 的引导器，而不是替代 MCTS，这个设计逻辑才是正确的。

三、_options()：每架无人机如何给候选动作打分？

有了 J_i，我们可以给每个**预期移动目标** q 打分。

报告 Q2 给出了：

$$\begin{aligned}\mathrm{Score}_i(q) = J_i(q) &+ 0.025\|q - g_i\|_1 \\ &+ \frac{0.3}{\max\{1, d(q)\}} + \mathrm{Risk}_{\mathrm{inter-drone}}(q).\end{aligned}$$

各项有什么含义？

项	用途	为什么不是额外的真实 Reward
$J_i(q)$	远期风险加权剩余距离	仅用于选择搜索候选
$0.025\|q - g_i\|_1$	稍微鼓励几何上接近目标	只是打破近似等价选择
$0.3/\max\{1, d(q)\}$	轻度排斥靠障碍较近的位置	只是启发式排序
机间风险项	减少联合移动时目标重叠风险	只影响提案策略

当前 `_options()` 会直接对越界目标、障碍目标施加很高的惩罚：

```
scores = np.full(self.base, INVALID, dtype=float)
for j, target in enumerate(targets):
    if in_bounds(target) and target_is_free(target):
        cost = self._cost(i, target)
        scores[j] = cost
        scores[j] += 0.025 * distance_L1(target, goal)
        scores[j] += 0.3 / max(1., clearance[target])
```

以上为依据真实代码简化后的教学片段。真实实现使用 NumPy 数组、缓存和具体的边界判断。

这里的“无效目标”指命令本身指向网格外或障碍格；即使命令目标合法，**随机偏移仍有可能把无人机带到障碍格**。算法没有把这些物理风险删除，而是让 `simulate()` 正常采样。

四、_proposal()：一架一架生成动作，怎么实现集中式协调？

现在来拆一段非常关键的代码：

```
cells = [tuple(map(int, p)) for p in state[:, :3]]
reserved = {cells[i] for i in np.flatnonzero(reached)}
active = np.flatnonzero(~reached)

if randomize or explore:
    active = self.rng.permutation(active)

for i in active:
    targets, cost, _ = self._options(int(i), state[i, :3])
    cost = cost.copy()
    for j, target in enumerate(targets):
        if target in reserved:
            cost[j] += RESERVED
```

这段代码背后的思想可以概括为：

先产生每架无人机的局部行动候选，再通过共享的占用信息构造一个尽量不冲突的联合动作。

1. 为什么初始化 reserved?

已经到达目标的无人机不会移动。这些格子必须视为被占用，否则其他无人机可能被引导进入同一格子。

2. 为什么每选完一架就把目标加入 reserved?

防止两个活跃无人机在同一次联合动作提案中，**预期**进入相同格子。这降低名义上的直接冲突，但不保证随机后继不会重叠。

3. 为什么有时随机打乱无人机处理顺序?

如果固定按编号顺序处理，第一架无人机总是先占据较好的位置，后面的无人机容易被系统性地牺牲。随机顺序能产生不同的联合动作组合，使 MCTS 有机会比较这些方案。

4. 机间随机风险如何加入?

`_slip_penalty()` 估计其他活动无人机在下一步可能因偏移到达候选格子的风险，用一个替代的惩罚向量影响排序。

重要的是：**它不是完整的未来联合转移分布，也不是一个硬安全证明**。真正的多无人机随机碰撞仍在 `_search()` 调用 `simulate()` 后体现。

五、Greedy、Sample、Explore 为什么要共存?

你的 `_proposal()` 不只有一种操作方式：

Greedy（确定性引导）：选启发式评分最低的联合动作作为强基线。它很快，通常有不错的导航能力。

Sample（偏向优良动作的随机提案）：在最多四个接近最优的动作中进行带权随机抽样。真实代码用 $e^{-3(\text{score}-\min)}$ 形成概率，且只保留距最佳分数不超过 1.05 的候选。

Explore（合法动作中的均匀随机探索）：在尽量不指向无效或已预留格子的前提下，增加动作多样性，避免搜索树只包含非常相似的动作。

对于 MCTS 的新动作扩展，当前参数 `random_action_prob=0.15`；对 Rollout，`rollout_epsilon=0.10`。这两个数不同，因为一个控制**树搜索的动作提案**，另一个控制**尾部模拟策略**。



图 8: Guidance 与 MCTS 的协作

六、_leaf()：搜索看到的不是完整未来，怎么办?

你在第二讲已经见过：搜索树深度有限、Rollout 最多八步。如果执行到截断处还没完成任务，需要估计后续价值。

真实 `_leaf()` 中：

```
distances = [
    math.inf if self._cut_off(i, state)
    else self._distance(i, state[i, :3])
    for i in range(self.count) if not state[i, 3]
]
```

这里的 `_cut_off()` 会检查**已经冻结在目标上的无人机**是否把其他无人机与目标隔开。

如果未来还剩 h 步，用 BFS 最少步数 δ_i 估计抵达目标的最早时刻。叶节点价值大致为：

$$\widehat{V}(s,h) = N c_{\text{step}} \sum_{j=0}^{D-1} \gamma^j + G \sum_{\substack{i:r_i=0 \\ 1\le\delta_i\le h}} \gamma^{\delta_i - 1}$$

其中 $D = \min\{h, \max_{i:r_i=0} \delta_i\}$，对于不可达的情况有专门处理。

这条公式包含两个直觉：

- 还需要走路，所以未来仍要支付步成本；
- 离目标较近的无人机，预计可以更早取得目标奖励。

但它是一个偏乐观的近似：它没有完整计入未来随机运动、障碍碰撞和复杂的协作等待。**叶节点启发式是 MCTS 可计算性的代价，而不是精确最优价值。**

七、一个深入的反例：为什么 Greedy 很强，MCTS 却可能犹豫？

你自己新增的 `supp_single_wallgap_2d` 场景，是一个典型教学案例。地图有一堵墙，只留下一个通道；无人机必须穿过那个区域。

在已有的固定 2,000 次搜索实验中，Full Solver 13/20 成功，Guided-only 14/20 成功。更值得注意的是，成功轨迹的平均步数分别约为 38.6 和 13.9。

这意味着：**MCTS 在当前设计下并不保证胜过一个已经很好的路线启发式。**

为什么可能出现这种现象？要区分两层：

1. 在真正走过的危险区域里，运动偏移可能导致撞墙，这是物理模型导致的风险。
2. MCTS 由于有限搜索深度、启发式尾部估值、Monte Carlo 噪声和根节点决策，可能在危险区域附近反复改变前进方向。这属于策略质量问题。

不能把第 2 类策略缺陷统统解释成不可避免的“理论上限”。是否已经达到理论上限，需要单独证明或提供可比较的最优策略界。

第三讲的核心结论

你其实使用了**两种截然不同的计算**：

$$\underbrace{\text{BFS / Dijkstra}}_{\text{提供近似路径信息}} + \underbrace{\text{MCTS}}_{\text{在随机环境中比较联合动作}}$$

前者让你有限的模拟更有方向；后者让你不必把真实的不确定性忽略掉。

请你自己回答：如果把 `_proposal()` 完全改成随机动作，为什么 UCT 公式本身没有变，但任务成功率仍可能显著下降？

第四讲：B3——从 plan() 到 step() 的 Python 闭环系统（8 分）

本讲是新增内容。对应提交入口 run_u8076655.py、environment.py、multi_drone.py、planner.py。B3 的正式得分来自现场程序表现。

我们已经建立数学模型并了解规划算法。下一步要回答：**一个数学算法究竟怎样变成在老师机器上能够运行、满足 10 秒限制的真实程序?**

一、首先区分三种程序角色

代码对象	真正职责	不应该混淆的事情
execution	保存本次任务的真实随机执行状态	不应该被树搜索反复推进
model	为规划器提供独立的 simulate() 随机后继	不是另一个需要完成任务的真实无人机系统
planner	在观测状态下输出一个联合动作整数	它并不直接修改执行环境

打开 run_u8076655.py，可看到：

```
execution = make_env(config, seed=seed, visualize=visualize)
state = execution.reset()

model = make_env(config, seed=1000003 + seed)
model.reset()

planner = MCTSPlanner(model, seed=2000003 + seed)
```

真实执行、规划时模拟、动作提案采用不同的随机数流。这个细节很专业：如果搜索消耗的随机数会影响真实未来，算法在做更多假想模拟后可能无意中改变测试任务的随机结果，削弱实验可比性。

因此：

$$\boxed{\text{真实执行 RNG} \neq \text{规划模拟 RNG} \neq \text{动作提案 RNG}}$$

这里的不同指随机数流被隔离，而不是随机运动规律不同。执行与模拟使用相同的转移机制、不同的采样序列。

二、一次 Episode 在代码中怎么走?

```
for step in range(cfg.max_num_steps):
    if success or failure:
        break

    action = planner.plan(state, planning_time)
    successor, reward, done, info = execution.step(action)

    total_reward += float(reward)
    discounted_return += cfg.discount_factor**step * float(reward)
    state = successor
```

把它翻译成自然语言：

1. 从真实环境获得当前联合状态；

2. 在有限预算内运行 `planner.plan()`；
3. 规划器返回一个整数编码的联合动作；
4. `execution.step(action)` 真正执行一次随机转移；
5. 记录真实奖励、新状态和是否成功；
6. 如果任务尚未结束，重复从第 1 步开始。

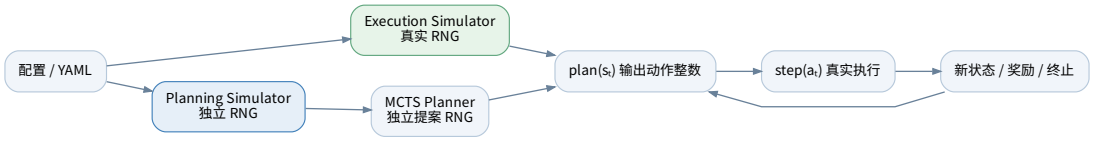


图 9: B3 双仿真器闭环

这就是在线规划系统最基本的闭环。`simulate()` 只是**假想未来**，`step()` 才是**真实任务推进**。

三、为什么代码中的 simulate() 还要知道剩余时间?

你的原始环境中，`step()` 会维护内部时间步并检查 `max_num_steps`；但是 `simulate(state, action)` 本身只模拟单次转移，并不负责推进真实时间计数。

因此，如果规划器在无限深度搜索中不处理剩余时域，就可能在假想未来里多模拟了本来已经没有的执行步数。

你在 `plan()` 中：

```
remaining = max(
    0,
    int(self.cfg.max_num_steps) - self.steps
)
```

以及 `_search(..., remaining, depth)` 中逐层递减 `remaining`，保证规划逻辑能够反映有限时域。

这也是第一讲“数学状态中要包含 t”在代码层面的落地。

四、10 秒规则不是一句口号，而是一个接口契约

老师 B3 的明确约束是：**每次实际决策最多 10 秒规划时间，不允许并行化**。

你的程序默认给每次规划：

$$\text{planning budget} = 9.5 \text{ s}.$$

在 `plan()` 中：

```
start = time.perf_counter()
budget = min(budget, self.budget_cap)
self.deadline = start + max(0., 0.97 * budget - 0.002)
```

也就是说，搜索不会主动用满全部 9.5 秒，而是预留一些时间作为安全余量。`_simulate()` 入口也会检查 deadline：

```
if time.perf_counter() >= self.deadline:
    raise Deadline
return self.env.simulate(state, action)
```

搜索到时间后，会中止当前试验并使用已完成的统计结果；如果没有完成试验，则返回已经准备好的 Greedy Fallback。

工程上要区分“主动检查截止时间”与“绝对硬实时保证”。如果某次模拟调用自身耗时很长、机器负载大，停止检查只能在函数调用前生效，不能在调用的中途任意打断。因此正式 Demo 仍应在老师的实验室机器上验证。

为什么还有 max_simulations = 2000?

一旦简单环境能够非常快地完成 2,000 次 Trial，就无需为了等满 9.5 秒而继续无意义地重复搜索。于是两个限制并存：

停止 = 达到搜索次数上限，或到达时间截止

而且一条 Trial 可能调用 `simulate()` 很多次，因为包含树内转移和 Rollout，因此“2000 Trial”不等于“2000 次随机运动采样”。

五、为何每个真实时间步都重建搜索根节点?

`plan()` 每次运行都会：

```
fallback = self._proposal(state, randomize=False)
root = Node()
```

这是典型的 **Receding Horizon / Replanning** 思想：观察新的真实状态后，在这个状态重新考虑未来。

优点是无需精确维护旧搜索树与新状态的一致性；缺点是没有直接复用上一步的 MCTS 统计数据。

在随机环境里，原来计划的轨迹经常不会原封不动地出现。在线重新规划可以应对这一事实。

六、从日志读取真实算法行为

报告 B3 使用 Open 2D 例子给出的部分日志：

```
step   1/80 | 0/1 at goal | reward    -1.0 | plan  9.14s  2000 sims | (1,3,0)
step   2/80 | 0/1 at goal | reward    -1.0 | plan  8.68s  2000 sims | (2,4,0)
...
step  11/80 | 1/1 at goal | reward    99.0 | plan  1.87s  2000 sims | (11,5,0)*
success: True (success), steps: 11, total discounted reward: 71.74
```

读懂每一列：

- `step`：真实环境已经执行的步数；
- `0/1`、`1/1`：已经到达目标的无人机数 / 总数；
- `reward`：本次真实转移产生的采样奖励；
- `plan`：本次规划耗时，而非全部 Episode 耗时；
- `sims`：完成的搜索 Trial 次数；
- 末尾坐标：本次真实移动后的无人机位置。

* 标记该无人机的 reached 状态。

这段日志尤其说明：**2000 次试验都用于决定当前的一个真实动作**。而下一时间步又进行新一轮搜索。

七、--visualize 与科研图表分别能看到什么？

当前代码有两类可视化。

真实执行场景（Vedo 交互图形）：显示障碍物、目标、无人机和实际执行轨迹。它回答“无人机最后怎么走”。

离线分析图：场景图集、风险热图、组件消融和预算曲线。它们回答“策略为什么这样设计，以及平均效果如何”。

你可以使用：

```
python run_u8076655.py \
  --config environments/demo_four_3d.yaml \
  --visualize
```

渲染窗口是解释工具，不应该被包含在 10 秒 MCTS 模拟的计算主循环里。

一条十分重要的实验伦理原则

不能把 MCTS 内部的假想轨迹当作真实 Episode 的结果。真正用于计算成功率、奖励和完成步数的，是 `execution.step()` 所产生的真实轨迹。

八、B3 的正确性还依赖哪些边界条件？

真实代码在进入规划循环前，检查目标是否重复、静态路径是否不可达。它还需要正确处理：

- 边界上的动作归一化：不能假设每个方向总是等概率合法；
- 到达目标后冻结：已完成任务的无人机仍然占据该格子；
- 静态障碍碰撞：真实模型可能让无人机留在原位置，但碰撞已导致任务结束；
- 无人机间碰撞：使用实际下一格占用而不是几何线段相交；
- 已达到步数上限：仿真 Episode 必须终止；
- 缺失可视化依赖：无图形环境下也应能以命令行运行。

这些属于 B3 的工程正确性；即使 MCTS 理论完全正确，忽略其中一项也可能导致现场失分。

第四讲总结

B3 的本质，是把数学规划算法封装成一个可测试、可重复、受时间约束的闭环决策系统。

你要能从代码中清楚指出三条界线：

$$\boxed{\text{预计算的静态启发式} \neq \text{搜索时的随机模拟} \neq \text{真实执行}}$$

如果老师问“为什么要有两个 Simulator”，请回答：**让规划器可以反复采样假想后继，同时保证真实任务的随机数流与规划计算隔离；这样性能比较才更可解释**。

下一讲，我们要进入 20 分的 B4：为什么科研实验不能只放一张成功轨迹，以及如何把“算法困难”变成可测量的统计问题。

第五讲：B4（20 分）——怎样像研究人员一样设计实验?

本讲是新增内容。对应作业 Part B Question 4、`A2/partB.tex` Experimental design 和 `A2_code/experiments.py`、`scripts/analyse_results.py`。

很多人把实验理解为：运行一下程序，报告“成功了”或者“用了多少步”。

但这是 Demo，不是科学实验。

真正的 B4 在问：**如果我们系统地改变问题复杂度，求解器究竟会怎样？哪一个设计组件真正有效？为什么?**

一、从一个科学问题开始，而不是从一张图开始

老师明确给了三个复杂度因素：

$$\boxed{\text{Complexity} = (\text{State Size}, \text{Action Size}, \text{Clutter})}$$

它们似乎都意味着“任务变难”，但作用机制完全不同：

- State Size：地图可出现的联合位置配置数量增加；
- Action Size：每次决策可选择的联合动作数量增加；
- Clutter：障碍物带来路径限制，并通过运动概率模型影响随机碰撞风险。

科研上最大的陷阱，是看见指标变化就马上给出因果解释。

例如，允许更多三维动作既增加了搜索分支数，也可能允许更快捷地绕开障碍。成功率改善时，你不能单凭观察就说“更多动作提高了搜索效率”。它也可能只是控制能力变强。

因此，一个可信的实验应当：

$$\boxed{\text{提出假设} \rightarrow \text{控制变量} \rightarrow \text{重复运行} \rightarrow \text{量化不确定性} \rightarrow \text{解释并限制结论}}$$

二、S1：为什么采用 3^3 全因子实验?

报告中 S1 的场景：

- 网格：$12 \times n_y \times 3$，$n_y \in \{5, 10, 20\}$；
- 每架动作数量：6、18、26；两架无人机的联合动作数：36、324、676；
- 障碍物数量：0、2、5，且使用嵌套的布局；
- 两架无人机，$\alpha = 0.25$，最大步数 $H = 60$；
- 每个条件重复 20 个执行随机种子。

因此：

$$3 \times 3 \times 3 = 27 \text{ 种条件}, \qquad 27 \times 20 = 540 \text{ 次 Episode}.$$

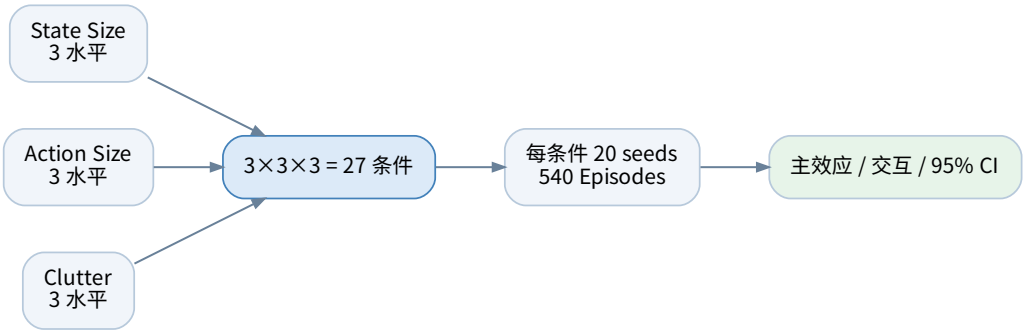


图 10: 三因素实验设计示意

这叫 **Full Factorial Design（全因子实验）**。你不只是分别改变一个因素，而是观察它们的各种组合。

为什么重要？因为复杂度因素可能互相影响。

例如，障碍物增加导致性能下降，但如果动作方向更丰富，这种下降也许会变小。这就叫**动作空间与 Clutter 的交互作用（Interaction）**。

1. 主效应是什么？

例如，比较 Clutter 从 0 到 5 时的平均惩罚完成步数变化：

$$\Delta L_C = \mathbb{E}[L \mid C = 5] - \mathbb{E}[L \mid C = 0].$$

其他因素要在比较中进行配对或平均，而不是随意混入不同环境。

2. 交互作用是什么？

设低动作空间时 Clutter 效应为 $\Delta L_C(A_{\text{low}})$，高动作空间时为 $\Delta L_C(A_{\text{high}})$。则可以比较：

$$I_{A\times C} = \Delta L_C(A_{\text{high}}) - \Delta L_C(A_{\text{low}}).$$

如果它明显不是零，就说明“障碍物的影响”取决于“动作空间是什么样”。

你不必把这个公式背成高等统计学定理，但必须明白：**三个因素的作用不一定可以简单相加**。

三、S2：为什么要做额外控制实验？

S1 观察到了一个反直觉现象：动作方向增多反而改善了完成表现。

一个严谨的研究人员不会马上说“动作越多越好”，而要问：

> 究竟是因为搜索分支变多了，还是因为机器人拥有更多实际移动方式？

S2 采用了两类控制实验。

1. 降低运动噪声

固定相关布局，将 α 从 0.25 调低至 0.01。

从

$$p_{\text{succ}}(p) = \frac{d(p)}{d(p) + \alpha}$$

可以看出，同一个 $d(p)$ 下，α 越小，预期移动成功概率越高。

如果低噪声时 Clutter 仍然增加完成步数，说明**几何绕行**本身就有成本。如果高噪声时障碍物造成的退化更大，那么**随机运动**可能进一步放大这种损失。

这比把“Clutter = 路程更长”写成唯一原因严谨得多。

2. 复制动作标签但不改变物理控制能力

例如原先只有 B 种真实移动指令，现在把每个物理方向在搜索接口中重复标记多次，让标签数增加，但执行仍对应原先的相同方向。

这样，你增加了**动作标签分支**，却没有增加无人机可实现的物理位移。

如果性能没明显变化，就不能宣称“动作数变多必定降低当前算法的效率”。对使用 Guided Progressive Widening 的规划器来说，大量额外标签可能根本不会在有限预算内被逐一展开。

注意，实验中的动作掩码和复制标签是**对照手段**，不是老师原始运动模型普遍规定的动作集合。

四、S3：为什么消融实验比“参数表”有用?

Ablation Study（消融实验）的核心问题是：如果把某个组件拿走，效果会怎样?

你有一个 Full Solver，其余变体只改变一组机制，尽量保持其他条件相同。

变体	修改内容	研究问题
`no_guidance`	使用均匀动作提案与较弱筛选	引导在何种复杂度下重要?
`no_leaf`	不使用截断叶估值	叶节点信息是否有帮助?
`no_widening`	不限制展开宽度	扩展规则是否提高效率?
`no_risk`	关闭风险加权项	风险感知在现有强引导基础上是否有效?
`visits_choice`	根节点改为访问次数优先	Anchored Root Decision 是否有价值?
`heuristic`	不执行 MCTS	搜索本身是否超过引导基线?

最容易忽略的逻辑陷阱：`no_guidance` 不只是去掉风险地图。它也改变了动作提案和可接受动作的过滤，所以如果 `no_guidance` 变差，最多说明“完整引导体系”很重要，不能将全部收益归功于风险加权 Dijkstra。

另外还需要研究：组件对困难和简单场景的作用是不是不同。例如，没有障碍物时，风险惩罚是否本来就不起作用? 拥挤环境中，它是否变得更重要? 这就是老师要求的 how each component affects performance as the problem becomes more complex。

五、S4：增加搜索时间，性能一定提高吗?

S4 用不同的 Trial Cap（例如 16、64、256、1024、2000）测试单机开放、单机强扰动、双机 Clutter、四机三维等场景。

你可以把它理解为：

> 一名规划器只思考一点点，与思考很久，最后真的会做出更好的决策吗?

理论上，更多采样可能降低某些价值估计的方差；但实际中的有限深度、候选动作偏差、启发式尾部估计、根节点切换机制，都可能使“增加搜索预算”不等于“显著提高任务成功率”。

因此 B4 要报告真实结果，而不是预先假设单调改善。

六、S5：为什么还要生成新地图?

S1 的全因子设计很适合比较几个主要因素，但地图模式和无人机数量有局限。因此 S5 构造 16 种组合：

$$N \in \{1, 2, 3, 4\}, \qquad \alpha \in \{0.05, 0.25, 1, 2\}.$$

每种组合选一个生成布局，运行 10 个随机种子，共：

$$4 \times 4 \times 10 = 160 \text{ Episode}.$$

它试图验证：结论是否能在另一批不同随机地图和更大联合动作空间中成立。

但要明白：**每个 (N, α) 只有一个布局，不能把不同配置的差异严格归因于无人机数量或噪声强度**。地图本身也可能不同，这是 S5 的重要解释边界。

七、3,300 次 Episode 具体从哪里来?

数据组	Episode 数量	主要用途
S1：全因子	540	三因素主效应与交互
S2：控制实验	160	区分几何、噪声与动作标签
S3：消融实验	880	搜索器内部组件的作用
S4：预算曲线	400	计算预算敏感性
S5：生成布局	160	不同无人机数和噪声水平
Regression	360	已知场景稳定性
Search-free Baseline	520	搜索和强引导之间的差异
Supplementary	280	缺口、共享走廊、高度层等极端案例
合计	**3,300**	**原始实验记录与汇总应一致**

一次 Episode 指的是从起点开始，一直执行到成功、碰撞或时间结束的一次真实任务运行。**并不是一次 MCTS Trial，也不是一次调用 `simulate()`**。

三个不同数量级必须分清：

$$\underbrace{3300}_{\text{实验 Episode}} \qquad \underbrace{2000}_{\text{部分实验每步 MCTS Trial Cap}} \qquad \underbrace{\text{上万次}}_{\text{一次规划可能产生的模型调用}}$$

八、科研统计的核心：为什么要有 95% Confidence Interval?

假设一个场景重复 20 次，每次记录最终惩罚完成步数 L_j。

样本均值：

$$\bar{L} = \frac{1}{n}\sum_{j=1}^{n} L_j.$$

样本标准差：

$$s = \sqrt{\frac{1}{n-1}\sum_{j=1}^{n}(L_j - \bar{L})^2}.$$

如果采用经典 Student-t 区间：

$$\boxed{\bar{L} \pm t_{0.975,n-1}\frac{s}{\sqrt{n}}}$$

这是在其适用假设下估计总体均值不确定性的一种方法。$n = 20$ 较小、结果可能有失败造成的双峰分布，所以区间只能视为近似。

1. 为什么成功率不直接用同一个公式?

成功率来自二元变量：成功是 1，失败是 0。小样本中经典正态近似可能表现不好，因此报告采用 **Wilson 95% Interval**。

不要把“13/20 = 65%”误解成“真实成功率精确等于 65%”。它只是当前随机种子的样本比例，存在抽样不确定性。

2. 为什么配对试验更有说服力?

比较 Full 与 No-guidance 时，如果让它们在同一地图、同一执行种子上运行，就可以对每组配对差值做统计：

$$D_j = L_j^{\text{ablation}} - L_j^{\text{full}}.$$

然后研究 $\bar{D}$，而不是把两个实验组简单当作不相关样本。

配对可以减少地图和随机条件带来的部分变化，但不意味着执行轨迹在两个不同策略下会完全相同。两个策略可能在前几步选择不同动作，因此即使共享随机种子，后续事件也会分叉。

3. “区间包含零”应该怎么说?

- 区间不包含 0：在当前估计与统计模型下，支持有非零差异；
- 区间包含 0：没有建立明确差异的证据；
- 区间包含 0 **不代表** 已经证明两个方法等价。

这是 B4 最容易被老师考问的统计措辞。

九、为何不能只比较完成步数?

设算法 A 在第 3 步撞墙，算法 B 在第 18 步成功。如果只看“Episode 步数”，就会荒谬地认为 A 更快。

因此定义 **Penalised Completion Length（惩罚完成长度）**：

$$\boxed{L = \begin{cases} \tau, & \text{成功}, \\ H, & \text{失败}. \end{cases}}$$

失败得到最大步数惩罚，避免“早死 = 高效率”。

同时最好保留 Success Rate、Discounted Return、Collision Count 和 Planning Time 等指标。因为 L 只是评估指标，**不是 B1 中的原始 Reward**。

十、实验有两套不同的“预算协议”

这个问题需要特别讲清楚。

B3 Demo：每一个真实决策步骤受 10 秒时间上限约束，默认规划预算 9.5 秒，且规划不并行。

B4 固定计算量实验：为了公平控制每次规划的模拟次数，会把 wall-clock 上限放宽，使用 2,000 Trial Cap 作为主要控制。复杂场景中单步可能超过 10 秒。

所以:

$$\boxed{\text{固定 Trial 数实验结论} \neq \text{严格 10 秒 Demo 条件下的性能证明}}$$

两个协议都有研究价值，但不能把其中一种的结果自动说成另一种。你现在的报告对此作了明确区分。

第五讲总结

一个严谨的机器人规划实验不只是“跑很多次”，而是:

先定义清楚要比较什么，再设计有控制的场景、重复采样、计算不确定性、验证算法组件的机制，并对不能支持的因果解释加以限制。

请试着回答:

如果允许更多移动方向后，任务变快，你能否断言“更大的动作空间使 MCTS 搜索更高效”?

正确的回答是：**不能直接断言**。因为更多方向同时增强了机器人本身的控制能力，必须使用类似 S2 的对照才能分开分析。

下一讲，我们拿起真实数据，一项一项解释 B4 究竟发现了什么，以及为什么某些结果与直觉相反。

第六讲：B4 的真实结果——怎样正确解释“有效”与“无效”？

本讲是新增内容，定量数据取自 GitHub 当前 `A2_code/results/summary.json`、原始实验 CSV 及 `A2/partB.tex`。小数四舍五入只用于教学展示，统计解释遵循报告定义。

研究实验的关键不是数据越多越好，而是：**你能否从结果中提取正确的信息，而不把未得到支持的判断当成结论**。

一、S1：三个复杂度因素的真实主效应

用惩罚完成长度 L 衡量任务性能，结果为：

因素：高水平减低水平	ΔL 均值	95% CI	我们能说什么?
State Size	$+0.02$	$[-1.40, 1.44]$	本实验没有发现明确主效应
Action Size	-5.54	$[-6.96, -4.13]$	更丰富的动作选择使 L 降低
Clutter	$+7.14$	$[5.68, 8.60]$	障碍物增加带来明确退化

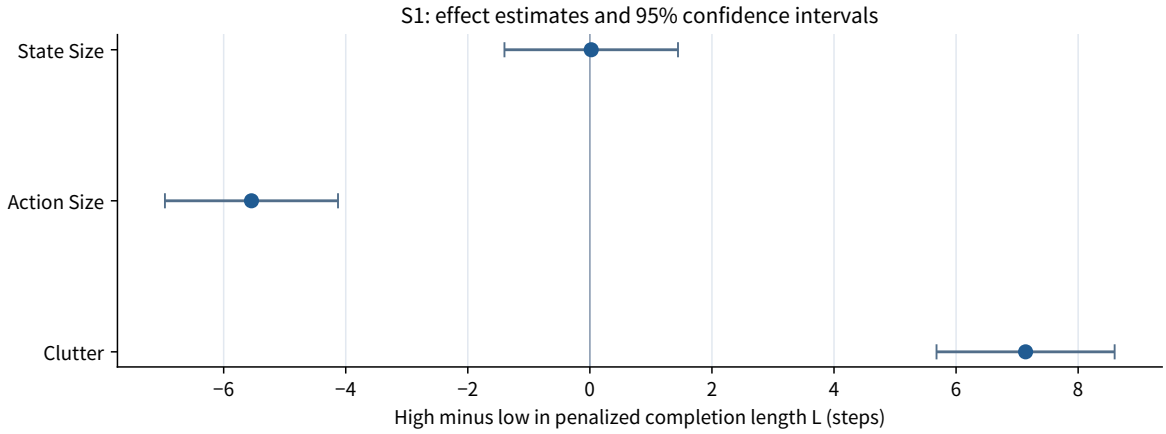


图 11: S1 三因素主效应

注意，正的 ΔL 是坏事：失败被赋值为 H，成功走得越久 L 越大；负值意味着整体表现改善。

1. 为什么状态空间扩大，效果却几乎为零?

很多人会说：“状态更多，规划器一定更难。”

这句话遗漏了**在线规划的局部性**。

MCTS 并不枚举全局所有状态。只要真实轨迹附近的局部动力学、目标距离和可达路径没有显著变化，给地图外侧增加很多不相关格子，未必就会明显增加在线计算难度。

你 S1 的 State Size 测试主要改变 n_y、增加可用空间，但并不等价于让无人机必须走更长的路径。因此正确表述是：

> 在这些固定起终点与地图设计下，额外离路线较远的状态空间，对有限预算的在线求解表现没有呈现明确影响。

不能推广成“状态空间大小对 MCTS 完全不重要”。当地图变大同时导致路线变长、岔路增多或协作更复杂时，情况可能不同。

2. 为什么动作空间扩大反而变好?

动作数从每架 6 个提高到 26 个，搜索分支确实增加。但是机器人同时有了更多的实际移动方向：斜向和竖向移动能够缩短路径、绕开障碍，也可能从偏移位置恢复。

因此：

$$\underbrace{|A| \uparrow}_{\text{搜索候选更多}} \quad \text{与} \quad \underbrace{\text{控制能力} \uparrow}_{\text{更易接近目标}} \quad \text{同时发生}$$

主实验里后者的收益超过了前者的代价，导致 L 下降。

S2 的重复动作标签控制进一步显示：仅仅增加等价的标签，并没有在当前测试中产生明确退化。**这说明不能用“动作数指数爆炸”的理论上界，直接预测所有实际任务一定退化**。

3. 为什么 Clutter 是最大的负面因素?

Clutter 至少通过两个机制起作用。

几何机制：障碍物迫使路径绕行，影响最短可达步数。

概率机制：仿真器让运动成功权重依赖到障碍物的距离。靠近障碍物时，随机偏移更显著，而偏移方向可能正好是障碍物格子。

因此障碍物增加可能同时降低控制的可预测性、增加碰撞机会、延长路径。

主实验中 $\Delta L_C = +7.14$ 的置信区间不跨零，支持这些测试水平下明确的性能退化。

4. 成功率层面是不是同样清楚?

新的 `summary.json` 还给出高减低水平的成功率变化（百分点）：

因素	成功率变化（pp）	95% CI
State Size	$+1.67$	$[-1.39, 4.72]$
Action Size	$+0.56$	$[-2.59, 3.70]$
Clutter	-5.00	$[-8.15, -1.85]$

注意区别：Action Size 明确降低了惩罚完成长度，但成功率变化的区间仍然包含 0。**不能因为平均完成表现变好，就声称成功率也显著提高**。

这正是为什么必须同时报告多个指标。

二、S3：真正的算法贡献在哪?

下面的数字是“移除或替换某一机制后，相对 Full Solver 的惩罚完成长度变化”：

消融 / 替代方案	ΔL	95% CI	解释
`no_guidance`	$+34.43$	$[31.47, 37.38]$	整体引导机制非常重要
`visits_choice`	$+3.37$	$[1.72, 5.02]$	当前根节点切换规则有实证收益

消融 / 替代方案	ΔL	95% CI	解释
`no_leaf`	$+0.03$	$[-1.45, 1.50]$	没建立明确贡献
`no_widening`	$+0.07$	$[-1.36, 1.50]$	没建立明确贡献
`no_risk`	-0.09	$[-1.39, 1.21]$	风险附加项收益未被证明
`heuristic`（无 MCTS）	-0.41	$[-1.72, 0.89]$	搜索增量价值尚不明确

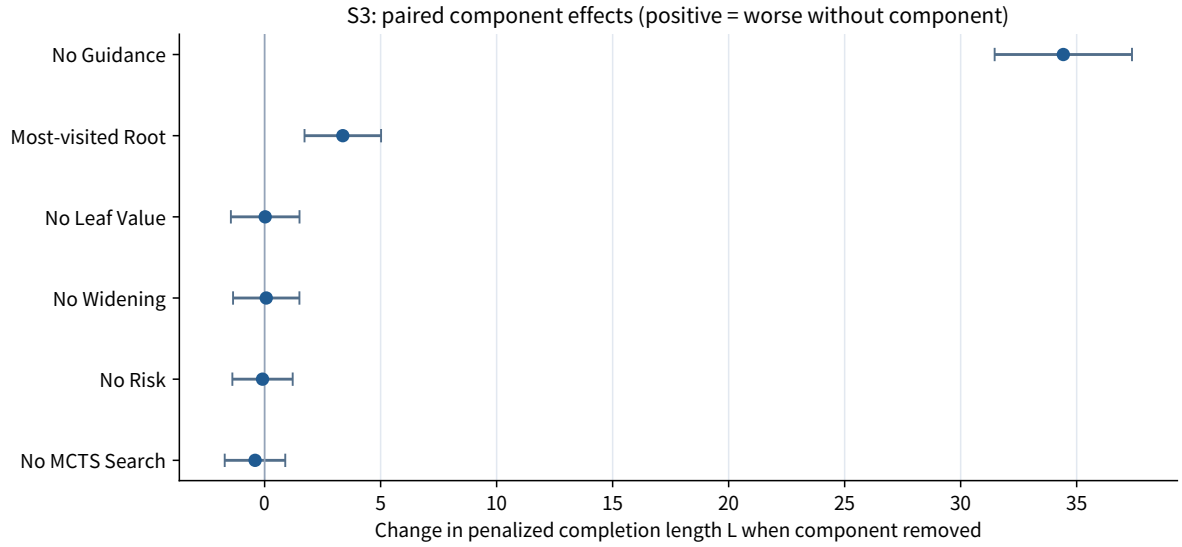


图 12: S3 组件消融主效应

1. 为什么 no_guidance 很差?

你当前 MCTS 在根节点只加入少量候选联合动作。把合理的动作提案换成均匀随机，会导致有限预算里大量采样落在不接近目标或可能危险的动作上。

这时，UCT 再精妙也无法凭空弥补没有进入候选集合的好动作。

因此，这个结果支持一句非常清晰的结论:

> **在动作空间巨大的在线规划问题中，候选动作的质量可能比搜索算法本身的复杂程度更关键。**

但同样不能把 +34.43 步全部归功于 Risk-aware Dijkstra：`no_guidance` 同时改变了候选动作分布和合法目标过滤。它是一个打包消融。

2. 为什么 Root Decision 有用?

常规 MCTS 可以选择访问最多的根动作。但你的 Guided Baseline 已经很强，有限采样下的动作价值可能受噪声影响。

Anchored Root Decision 相当于提出一个问题:

> 其他动作的 Q 值真的比 Greedy 好到足以抵消目前的采样波动吗?

如果没有，就继续执行 Greedy。实验中相对于 Most-visited，这一规则改善 L，且区间不跨 0。

但根节点统计样本不是传统意义的独立同分布样本，因此“方差阈值”不应被称为具有严格一类错误控制的显著性检验。

3. 为什么 Leaf / Widening / Risk 没有明确效果?

这不是“算法有错”的证明。

它可能意味着：在此次测试范围、强引导基线和固定搜索预算下，组件所解决的问题并没有成为主要瓶颈；也可能说明该组件带来的好处被某些偏差抵消。

例如，叶节点价值依赖 BFS 最短距离，可能对复杂绕行或者随机碰撞风险过于乐观。增加一个不准确的估值不一定提高实际决策质量。

科研报告不是要求你证明每个组件都有益，而是要求你正确识别哪些效果有证据、哪些还没有证据。

4. 最重要的负面结果：MCTS 未必优于 Guided-only

S3 中取消搜索得到 $\Delta L = -0.41$、区间 $[-1.72, 0.89]$; 这既不证明 MCTS 更差, 也不证明它与 Guided-only 等效。

可以严谨地说:

对于目前评估过的这些场景与预算，尚没有证据证明完整 MCTS 相对于强引导策略稳定改善了 L。

这为什么很有价值? 因为它揭示：算法中的主要性能来源，可能是 Guidance，而非搜索本身。这是理解系统瓶颈、设计下一步研究的真实起点。

三、S4：更多搜索是否带来更多收益?

S4 用 16、64、256、1,024、2,000 Trial Caps，在多种复杂度的场景中比较成功率和 L。

报告的结论是：**没有观察到从 16 到 2,000 次模拟的一致单调改善。**

你需要理解三个原因。

第一，**搜索深度增加，不代表价值估计就变得准确**。Leaf Value 本身仍可能有系统偏差。

第二，**更多 Trial 会探索更多动作**，有时反而把有限样本分散到新的候选方向，而不是只反复评估最佳动作。

第三，真实环境存在不可避免的执行随机性，好的价值估计不能控制实际采样到的每一次风扰动。

因此，实验可以观察到搜索树更深、候选动作更多，却没有同等幅度的成功率改善。

这也解释了：**2000 次模拟是计算预算参数，不是算法最优性的证明。**

四、S5：四机强噪声为什么尤其困难?

S5 的 160 条生成布局测试中，汇总成功 148/160，整体成功率为 92.5%。但这一平均值掩盖了不同配置的差别。

例如 `random_15_N4`（四机、$\alpha = 2$）的 10 个执行种子只成功 3 次；7 次失败中，6 次涉及静态障碍碰撞，另 1 次涉及无人机间碰撞。

这并不必然说明 MCTS 选择错误。它可能包含无法通过换动作消除的运动风险，也可能包含启发式候选动作不足、搜索质量与多无人机随机结果组合爆炸造成的损失。

尤其重要的是：**S5 每种 (N, α) 配置只有一个地图，不足以把这一组低成功率直接归因于“所有四机强噪声任务”。**

正确表述是：这是所测试的具体布局在该噪声水平下的困难表现。

五、补充压力测试：墙壁缺口与共享走廊

你新增的 Supplementary Case 很适合作为算法失效机理的案例。

场景	Full MCTS 成功	Guided-only 成功	需要深入解释的现象
单机墙壁缺口	13/20	14/20	搜索可能反复绕行，在危险区停留更久
双机共享走廊	13/20	10/20	目标冻结与路径连通性造成协作问题
单机五障碍	20/20	15/20	搜索有时能改善特定布局的成功表现
双机不同高度层	20/20	19/20	可以利用不同高度降低名义冲突
四机不同高度层	20/20	20/20	这组简单分层布局不是主要瓶颈

其中墙壁缺口和共享走廊分别有 7/8 个障碍物，超出了 B3 主要 Demo 描述的“最多 5 个障碍物”，因此它们应理解为**额外压力测试**，而不是对老师隐藏 Demo 的直接预测。

成功轨迹的平均步数不能替代整体表现分析：失败轨迹提前终止，比较“只在成功条件下的平均步数”会带来选择偏差。对于方案比较，应继续使用 L、成功率和折扣回报。

六、你应该怎样写出有说服力的结论?

一段强结论通常具有四层结构:

1. **事实**：报告观测到了什么数字或图形?
2. **统计**：区间是否支持明确效应？样本数量够不够?
3. **机制**：哪个算法机制能合理解释它?
4. **限制**：有哪些混杂因素或未被验证的其他解释?

例如，对 Clutter 的解释可以写成:

> 在本实验设定下，Clutter 从 0 到 5，使惩罚完成步数平均增加 7.14，95% CI 为 [5.68, 8.60]。运动模型中较小的障碍距离会降低预期动作成功权重，障碍还可能要求更长的绕行。因此，几何绕行和随机偏移风险共同构成合理解释。不过这些结果仅适用于本实验的障碍布局与控制设置，不能据此直接推广到任意地图。

这就是研究型作业应有的严谨程度。

第六讲总结

你的实验真正证明的是：Guidance 对当前规划器特别重要，Root Decision 有可观察收益，Clutter 是测试范围内最明显的退化因素；而 MCTS 相对 Guided-only 的稳定增量价值，仍然是未解决的问题。

请试着回答:

> 为什么 `no_guidance` 显著更差，并不自动证明“风险加权 Dijkstra”本身具有显著收益?

正确思路是：消融同时改变多个引导机制，要通过 `no_risk` 等更精细对照来拆分贡献。

下一讲我们讨论更深的问题：不确定环境到底存在怎样的理论上限？我们又如何区分物理上不可能和算法还没有做好?

第七讲：理论上限、失败机制与可以证明的结论

本讲是新增内容。结合 B1 的转移模型、S3/S5/补充实验、随机过程基础。重点建立“实验表象”和“数学证明”的区别。

Claude 提到“已经达到理论上限”，这句话在机器人规划研究中必须慎重使用。

它可能包含一个正确直觉：**在随机运动模型中，机器人不可能完全控制未来发生什么**。但它也可能包含一个未被证明的推断：**当前规划器已经没有任何改进空间**。

这两句话在逻辑上不是同一回事。

一、成功率的最优策略，与奖励的最优策略并不一样

你 B1 定义的目标是：

$$\pi_R^* = \arg\max_\pi \mathbb{E}_\pi \left[\sum_t \gamma^t r_t \right].$$

但如果只考虑任务成功概率，另一项目标是：

$$\pi_P^* = \arg\max_\pi P_\pi(\text{success}).$$

一般而言：

$$\boxed{\pi_R^* \neq \pi_P^* \text{ 是可能的}}.$$

为什么？因为奖励还包含尽快抵达目标的收益和延迟成本。某些更快、风险略高的路线，可能获得更高期望折扣回报；另一些更慢路线，可能成功率更高。

因此，在讨论“理论上限”之前，必须先说清楚：你要证明的是**最大奖励**还是**最大成功率**。

二、如何在数学上给成功率最优值下定义？

对于有限时域且每步状态、动作有限的系统，可以将成功概率作为终止收益，通过动态规划求：

$$P_h^*(s) = \max_a \sum_{s'} T(s,a,s') \big[\mathbf{1}\{s' \text{ 为成功终止}\} + \mathbf{1}\{s' \text{ 为未终止}\} P_{h-1}^*(s') \big].$$

碰撞与超时终止的成功收益为 0；成功终止的收益为 1。

这条方程和 Bellman Optimality 在结构上相似，但研究的是**成功概率**而非原始奖励。

如果状态和动作空间很小，可以精确计算这一最优成功率。这能用来严格验证一个规划器离最优有多远。

但在四架无人机、高维三维网格中，穷举通常难以实现。此时只能尝试构造界、使用更强基线或缩小问题规模求解，而不能从有限样本成功率直接宣布已经最优。

三、为什么狭窄通道可能具有不可避免风险？

假设所有可行策略都必须连续经过 m 个高风险位置。并且对每一次经过的条件碰撞概率，都可以证明至少为 $\rho > 0$。

那么每一步在这些关键位置存活的条件概率至多为 $1 - \rho$；通过连乘，有：

$$\boxed{P(\text{成功通过}) \le (1-\rho)^m}$$

这是**带有前提的理论上界示例**，不是对你具体地图已经计算出的上界。真正用于你的地图时，需要先证明任何成功轨迹都必须承担相应风险，并处理状态与动作引起的条件概率变化。

如果无法建立这样的下界，就不能直接套上式宣称当前 65% 成功率已经达到数学上限。

四、单步概率本身还有“方向不敏感”的微妙结构

在你仿真器的转移模型中，排除越界后，预期方向获得归一化成功权重，其余方向分享归一化偏移权重。

如果从同一个位置 p 出发，比较两个都合法、都非障碍的预期方向，那么在当前位置发生随机偏移撞障碍物的概率，基本由附近障碍方向数和偏移概率决定；换一个合法方向，单步风险不一定改变。

因此，风险引导的有效作用主要在于：**让未来经过的格子更加安全、缩短风险暴露时间、避开持续贴墙的路线**。

这也解释了墙壁缺口测试中的一个事实：反复在墙边徘徊，未必比果断通过更安全，反而可能因为风险暴露次数增加而更容易碰撞。

五、把三类困难场景分开理解

1. 单机墙壁缺口：策略犹豫与物理风险叠加

场景中只有一个通道可以穿越。失败的记录主要是障碍碰撞；成功轨迹有时还出现明显的反复绕行。

你可以提出三个竞争性解释：

- 受约束的地形让任何可行路径都必须靠近障碍，存在不可避免的单步风险；
- 近似 Leaf Value 可能错误估计了短期后退与长期完成任务之间的价值；
- 有限 Monte Carlo 回报、候选动作偏差与根节点切换规则可能让策略在瓶颈附近反复改变方向。

现在只能说这些解释是**与观测相容的机制假设**，不是已经排除其他因素的因果证明。

2. 双机共享走廊：到达目标不再是纯粹的好事

一架无人机到达目标后永久冻结。如果目标位于另一架无人机的必经通道中，过早完成一个局部任务，可能损害整体任务的可行性。

这是一个典型的**多智能体耦合、任务顺序**问题。

例如，A 要穿过 B 的目标格子才能到达远端目标。理想的协调顺序可能是 A 先穿过去，然后 B 再落到自己的目标上。

你现有 `_cut_off()` 检查已经冻结的目标占用是否阻断路径；但是它不自动保证每次都能提前预测并执行最理想的完成顺序。

重要的事实限制：当前共享走廊 Full Solver 的 7 个失败记录都涉及静态障碍碰撞，并没有直接证明“提前冻结造成不可达”就是这 7 次失败的具体原因。

3. 四机强噪声：风险乘积与联合状态爆炸

单架无人机面临一定风险时，多架无人机的任务要求**全部**到达、任意碰撞都使任务结束。

如果在某一步，各架无人机条件独立且每架撞障碍概率均为 p，则至少一架撞障碍的概率为：

$$P(\text{至少一架撞障碍}) = 1-(1-p)^N.$$

例如 $p = 0.08, N = 4$:

$$1 - (0.92)^4 \approx 28.4\%.$$

这个数字只是用于说明风险累积的假设性例子，不是特定 `random_15_N4` 地图的实测概率。

联合任务的成功要求比单机更苛刻，碰撞风险又会随随机结果组合增长，这是四机强噪声测试困难的重要机制。

六、理论上不可能，与算法没找到好策略，不是一回事

观察	可以得出的结论	不能自动得出的结论
多次随机碰撞	任务存在风险	已达最优成功率
MCTS 比 Greedy 慢	当前搜索可能浪费计算	任何 MCTS 都没有价值
某个消融置信区间跨零	尚无明确差异证据	两个算法严格等价
2,000 Trial 没改善	当前预算实验未见一致收益	更多任何形式的计算都无用
四机某布局 3/10	该布局在当前设置下很困难	所有四机强噪声环境都只有 30% 成功率

七、如何用课程内方法继续研究?

你不需要立刻添加课外的复杂算法。仍可以围绕老师教过的核心结构进行验证:

- 检查强引导 Greedy 在这些场景中是否已经很接近可能达到的表现;
- 比较不同 UCT 探索系数、Rollout 深度、候选动作提案与根节点规则;
- 构造足够小的地图，使用 Bellman 动态规划或穷举得到最优成功率，作为严格的参照;
- 在严格相同的执行预算下做配对实验，而不是只看某个随机种子的成功轨迹;
- 不修改老师定义的原始 Reward 或随机转移规则，只改规划方法。

这些都保持在 **Model-based Online Planning** 的课程范围内。

第七讲总结

当你理解以下这句话时，就已经触及这份作业的理论核心:

> **一个随机机器人系统的失败，可能来自环境随机性的不可避免风险，也可能来自有限计算和近似策略导致的次优决策。仅凭失败次数不能区分两者。**

请试着回答:

> 为什么“某条狭窄走廊必然有撞墙风险”不能直接推出“你的 MCTS 13/20 已经达到理论最优”?

答案的逻辑是：虽然证明了风险不可能降到零，但还没有计算在所有允许策略中的**最小风险或最大成功概率**，因此仍存在潜在策略改进空间。

第八讲：把整份 Part B 讲给考官听——代码地图、口试问答与知识结构

本讲是新增内容，目的是把前七讲转化为可以独立解释、能够现场运行且经得起追问的知识。

到这里，你已经知道 Part B 里的模型、算法、程序和实验。最后一个问题是：**如果老师从你的报告中随机挑一个公式、一段代码或一张结果表，你能否在不背稿的情况下解释它?**

真正掌握的标准，不是复述术语，而是能够回答“为什么不是其他做法”“它依赖哪些假设”“这项实验究竟证明了什么”。

一、把整个项目压缩成一张知识结构图

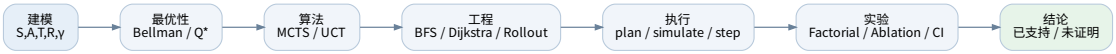


图 13: 八讲知识树

你可以把整个项目理解成四层推导:

$$\begin{aligned} &\text{数学问题：} && (S, A, T, R, \gamma), \\ &\text{理论目标：} && \max_a Q^*(s, a), \\ &\text{近似算法：} && \text{UCT + Guided MCTS}, \\ &\text{实证证明：} && \text{paired experiments + uncertainty}. \end{aligned}$$

第一层定义问题，第二层告诉你什么叫最优，第三层告诉你怎样在有限时间内近似求解，第四层检验这个近似求解器是不是真的有用。

二、最重要的代码阅读地图

文件或函数	阅读目标	推荐联系哪一讲?
`multi_drone.py::_transition_dynamics()`	真实随机转移与 Reward	第一讲
`planner.py::__init__()`	参数、地图预计算、归一化	第三讲
`_slip_model()`	从已知动力学估计局部风险	第三讲
`_distance_map()`	BFS 最少物理步数	第三讲
`_cost_map()`	风险加权 Dijkstra	第三讲
`_options()`	每架无人机的候选动作分数	第三讲
`_slip_penalty()`	其他无人机的局部随机占用风险	第三讲
`_proposal()`	拼装一个联合动作提案	第三讲
`_leaf()`	截断处的近似未来价值	第二、三讲
`_rollout()`	用启发式模拟未来	第二讲
`_search()`	MCTS 四阶段主循环	第二讲

文件或函数	阅读目标	推荐联系哪一讲?
`Edge.add()`	Q 均值与方差的增量更新	第二讲
`_anchored()`	最终根动作决策	第二讲
`plan()`	每步时间预算与最终动作输出	第二、四讲
`run_u8076655.py::run_episode()`	真实执行与数据记录	第四讲
`experiments.py`	实验场景、变体和重复运行	第五讲
`scripts/analyse_results.py`	置信区间、配对效应和画图	第五、六讲

建议阅读顺序不要按照文件从第一行到最后一行。先读仿真器的 `step/simulate`，再读 `plan/_search`，随后才回头看动作引导和叶估值。这样每一行代码都有目的，不容易迷失在参数和缓存里。

三、15 个最值得准备的口头问答

Q1. 为什么用 MDP 而不是简单的最短路?

因为实际运动结果具有随机性。最短路基于确定性边代价，而 MDP 通过 $T(s, a, s')$ 和奖励函数，比较采取动作后的所有可能后继与未来回报。BFS/Dijkstra 在本系统里只负责提供启发式。

Q2. 既然转移模型已知，为什么不直接 Value Iteration?

因为联合状态和动作空间随无人机数量迅速增长，精确枚举和动态规划代价过大。在线 MCTS 只在当前状态周围进行采样，直接调用生成式 `simulate()`，将有限预算集中到候选动作。

Q3. 为什么不使用 POMDP?

本题将所需的无人机联合位置、reached 标记和时间信息视为已知，随机运动规律也已给定。这里的困难是**随机转移**，不是必须推断隐藏状态。随机性并不等于部分可观测性。

Q4. 为什么状态还要包含 reached 和时间?

到达目标后的无人机会冻结并占据格子，首次到达还有一次性奖励；时间决定还剩多少执行步数。不区分它们会失去预测未来转移与奖励所需的信息。

Q5. 运动扰动独立，为什么还要联合规划?

即使各无人机的原始运动方向在给定当前状态和动作下条件独立，碰撞事件和任务整体终止条件仍然耦合。多个无人机可能最终占据同一格子。

Q6. Selection、Expansion、Simulation、Backpropagation 分别是什么?

Selection 使用 UCT 分配已加入动作的搜索预算；Expansion 通过渐进拓宽和动作提案添加候选动作；Simulation 使用启发式 Rollout 估计未展开的未来；Backpropagation 用采样回报更新动作价值均值、访问次数与离差平方和。

Q7. 为什么 UCT 会探索当前 Q 比较低的动作?

因为它的第二项 $c\sqrt{\ln N / N_a}$ 鼓励访问少的动作。这样避免仅凭初期少量样本就认定某个动作永远最好。

Q8. 为什么 Progressive Widening 有意义?

四架无人机最多有 456,976 个联合动作，有限 Trial 根本无法逐一评估。渐进拓宽让节点访问次数增加时才逐渐扩展候选动作，减少无效展开，但可能错过未被提出的高价值动作。

Q9. 为什么还需要 Risk-aware Dijkstra?

BFS 只考虑最少步数，Risk-aware Dijkstra 在路径成本中加入由已知转移模型计算的局部偏移撞障碍风险。它用于候选动作选择和 Rollout 引导，不能代替原始 MDP 的 Bellman 价值函数。

Q10. 为什么 Rollout 不完全随机?

随机 Rollout 在巨大联合动作空间中往往产生大量不接近目标的轨迹。启发式 Rollout 能让有限模拟更常接触与任务完成相关的未来，降低估值的无关噪声。但它也可能引入模型偏差。

Q11. 为什么要有 Leaf Value?

树深和 Rollout 长度有限。如果在截断时把所有未终止状态的未来价值都设为零，会低估距离目标很近的状态。Leaf Value 用 BFS 剩余步数近似步成本和未来目标奖励，但不是精确价值。

Q12. 为什么使用 Anchored Root Decision?

强 Greedy 已经提供可靠基线。MCTS 估计有随机波动，因此只有其他动作估计的优势大于一个按样本离差缩放的阈值才切换。这是启发式稳健决策，不是严格统计检验。

Q13. 规划模拟为什么不能使用同一个真实环境 RNG?

因为树搜索会调用 `simulate()` 很多次。如果这些调用与真实执行共享同一个随机数流，就可能因为搜索量不同而改变后续真实风扰动的采样序列。分离 RNG 提高实验控制和可复现性。

Q14. 为什么动作空间增大，B4 结果反而改善?

动作数增加的同时，机器人获得了更丰富的移动方向，可能减少绕行和恢复步数。不能把这个效果直接归因于搜索更高效。S2 通过复制动作标签、固定物理运动能力帮助分离这两种机制。

Q15. 为什么 Full MCTS 没显著超越 Guided-only，还能说设计合理?

老师要求的是一个具体的近似 Online MDP Solver 及其真实实现和实验分析，不要求每个组件都显著改善每种指标。当前 Guidance 的贡献具有强证据，Root Decision 有实证收益；MCTS 的增量价值在这些预算/场景下尚未建立。坦率陈述局限比虚构优势更符合科研标准。

四、现场 Demo 的 90 秒解释框架

你可以按照这个顺序组织语言，先说明问题和设计目标，再解释关键实现，最后给出局限：

> 本任务是一个集中式有限时域 MDP：状态包含各无人机的位置、是否到达目标以及时间。动作是多架无人机的联合移动，实际转移受到运动扰动，碰撞导致终止。我选择在线 UCT/MCTS，因为联合动作空间大，而且仿真器能够从已知转移模型直接采样后继。
>
> 每个真实时间步，我先根据 BFS 与风险加权 Dijkstra 产生目标导向的联合动作提案，再通过 Progressive Widening 控制候选集合，用 UCT 分配模拟预算，使用启发式 Rollout 与叶节点估值计算回报，最后利用 Greedy-anchored 的根节点规则返回动作。真实执行与搜索模拟使用独立随机数流，每步规划受到 10 秒限制。
>
> 在实验中，我系统改变状态、动作与障碍复杂度，并通过重复运行、配对消融和 95% 置信区间分析各部分作用。最明确的发现是障碍物增加导致退化，整体 Guidance 很重要；但在现有测试里尚未证明完整 MCTS 稳定优于不搜索的强引导基线。

这段话是**理解后的示范口头说明**，不应机械背诵。考官如果追问某一项，你还需要能回到前七讲给出公式和代码依据。

五、12 条公式索引：一眼看到知识主线

1. MDP 定义

$$\mathcal{M} = (S, A, T, R, \gamma).$$

2. Markov Property

$$P(s_{t+1} \mid s_{0:t}, a_{0:t}) = P(s_{t+1} \mid s_t, a_t).$$

3. 联合动作规模

$$|A| = B^N, \quad B = 8 \text{ 或 } 26.$$

4. 转移模型

$$T(s, a, s') = \sum_{y:F(s,y)=s'} \prod_i q_i(y_i \mid p_i, r_i, a_i).$$

5. 随机运动成功权重

$$p_{\text{succ}}(p) = \frac{d(p)}{d(p) + \alpha}.$$

6. 采样奖励

$$\tilde{r} = N c_{\text{step}} + C(K_O + K_V) + G \sum_i \Delta r_i.$$

7. Bellman Optimality

$$V_h^*(s) = \max_a \mathbb{E}[r + \gamma V_{h-1}^*(s') \mid s, a].$$

8. UCT

$$a = \arg\max_a \left[Q(s, a) + c \sqrt{\frac{\ln(N(s) + 1)}{N(s, a)}} \right].$$

9. Progressive Widening

$$K(s) = \left\lceil 0.5 \sqrt{N(s) + 1} \right\rceil.$$

10. 风险启发式

$$J_i(p) = \min_{\pi: p \rightsquigarrow g_i} \sum_{q \in \pi} (1 + \lambda \rho(q)).$$

11. Monte Carlo Backup

$$Q_{\text{new}} = Q_{\text{old}} + \frac{G - Q_{\text{old}}}{n_{\text{new}}}.$$

12. 惩罚完成长度

$$L = \begin{cases} \tau, & \text{成功}, \\ H, & \text{失败}. \end{cases}$$

这 12 条式子不是互不相关的考点，而是构成了从**问题建模、最优性、近似求解到实证评价**的完整链条。

六、六道理解检验题（附答案提示）

1. 如果一个动作导致随机偏移，但这次仍然成功抵达目标，算不算规划器选择正确？——一次轨迹成功不能证明动作期望价值最优；要区分采样结果与期望质量。
2. 为什么 `simulate()` 调用次数通常远大于 MCTS Trial 次数？——一次 Trial 内包含树内多个边与 Rollout 多步。
3. 为什么 MCTS 增加搜索次数以后未必更好？——有偏启发式、候选动作扩展和随机回报都可能限制增益。
4. 如果 `no_risk` 与 Full 没显著差别，能否删除整个 Guidance？——不可以；风险项与完整的路线/动作引导不是同一个组件。
5. 为什么增加障碍物有时既影响几何又影响转移概率？——老师的 $d/(d+\alpha)$ 显式依赖障碍距离。
6. 如果你想证明“13/20 已达理论上限”，最低限度还需要什么？——为这个具体场景求或界定最优成功概率，区分奖励最优与成功概率最优。

第八讲总结

如果你只记住一个观点，那就记住：

> **好的智能机器人规划工作并不是把很多算法名称拼起来，而是以严格的随机决策模型为起点，解释计算为何困难，再设计一个受约束的近似算法，最后用可复现的实验去检验它与自己的假设。**

这正是 COMP8620 Part B 在研究生层面真正考查的能力。

附录 A：原题要求与课件定位

原题文件：a2.pdf 第 3 页，Part B [35 pts]。其核心要求如下：

题目	分值	精确要求的类型	对应讲次
B1	3	给出多无人机随机导航的 MDP 定义	第一讲
B2	4	提出具体的近似在线 MDP Solver 与伪代码；若用 MCTS，说明四阶段	第二、三讲
B3	8	用 Python 基于给定功能实现，现场测试两组场景；每步最多 10 秒、无并行规划	第四讲
B4	20	系统比较状态、动作、障碍复杂度；重复运行、均值与 95% CI；分析组件表现	第五、六、七讲

原题特别强调 B1–B4 必须一致：B3 实现 B1/B2 所描述的模型和算法；B4 分析必须来自真实代码运行。

主要课程材料：

- week5-MDPSolving.pdf：MDP 求解、RTDP/LRTDP、Online Solving、MCTS 四阶段、UCT、Heuristic Simulation 与 Backpropagation。
- week6-MDPSolvingCont.pdf：在线规划与 MCTS 的延续讲解。
- MCTS.pdf、OnlineMDPSolving-RTDP-LRTDP.pdf：与课堂方法相关的补充论文和阅读材料。
- week8-RLBasics.pdf、week9-RLBasics-1.pdf、week10-RL.pdf：强化学习背景；本作业核心仍是已知转移模型的 Online MDP Planning，不应把 DQN、PPO、SAC 误写成提交的主算法。

GitHub 实现（本讲义所依据版本）：https://github.com/lrlorry/8620，main，核对时最新 commit 为 3a6f4de051f166ea2742333e808f1c973818585b。

重要文件直达：

- Part B 报告原文
- 主规划器 planner.py
- 真实执行入口 run_u8076655.py
- 仿真器 multi_drone.py
- 实验汇总 summary.json
- 实验运行程序 experiments.py
- 实验结果分析脚本

引用与内容界限：前两讲保留本对话原先讲授内容；第三讲之后为进一步讲解、推理和教学示例。凡涉及具体实验数字，均按照对应 GitHub 实验汇总与报告解释；数学上的上界示例和启发式成本算例属于教学推演，不应当被当作原始实验测量。

附录 B：完整版学习检查清单

- □ 能独立写出联合状态 $s = ((p_i, r_i), t)$ 并解释每一项的必要性。
- □ 能区分未归一化运动权重与真实概率分布。
- □ 能从独立运动采样解释整体碰撞耦合。

- □ 能解释 Reward 与单独成功率目标为什么不同。
- □ 能写出 Bellman Optimality 并解释计算规模。
- □ 能区分 `simulate()` 与 `step()`，并说明独立 RNG 的作用。
- □ 能解释 UCT 的 Exploration 与 Exploitation。
- □ 能推导 26^N 和 Progressive Widening 的用途。
- □ 能解释 BFS、风险加权 Dijkstra 与真正 MDP Value 的区别。
- □ 能解释 `_proposal()` 为何影响 MCTS 表现。
- □ 能解释 `_rollout()`、`_leaf()` 与 Backpropagation 的价值估计。
- □ 能解释为何根节点执行规则不同于树内 Selection。
- □ 能解释每步 10 秒预算、2,000 Trial 与生成式模型调用数的关系。
- □ 能复述 S1–S5 每组实验的研究目的。
- □ 能解释配对差值与 95% 置信区间的含义。
- □ 能解释 Clutter 和 Action Size 的非直觉结果。
- □ 能如实描述 Guidance 的强证据和 MCTS 增量价值的不确定性。
- □ 能将“随机环境固有风险”与“算法策略次优”区分开来。

讲义终。适用于逐章学习、公式复习与 COMP8620 A2 Part B 现场答辩准备。